

PLANT DISEASE DETECTION USING YOLO v8 ALGORITHM

A PROJECT REPORT

Submitted by

NITHISH KUMAR.G

PRASHATHAN.V

ROHITH JOSHUA.M

SUBRAMANIAN.I

In partial fulfilment for the award of the degree

of

BACHELOR OF ENGINEERING

in

ELECTRONICS AND COMMUNICATION ENGINEERING



SARANATHAN COLLEGE OF ENGINEERING
(An Autonomous Institution)
Tiruchirappalli 620 012



ANNA UNIVERSITY : CHENNAI 600 025

APRIL 2025

SARANATHAN COLLEGE OF ENGINEERING
(An Autonomous Institution)
Tiruchirappalli 620 012

ANNA UNIVERSITY: CHENNAI 600025

BONAFIDE CERTIFICATE

Certified that this project report **Plant Disease Detection using YOLO v8 Algorithm** is the bonafide work of **Nithish Kumar.G, Prashathan.V, Rohith Joshua.M, Subramanian.I** who carried out the project work under my supervision.

SIGNATURE

Dr. M. SANTHI

HEAD OF THE DEPARTMENT

Professor

Department of Electronics &
Communication Engineering
Saranathan College of Engineering
Tiruchirappalli-620012.

SIGNATURE

Dr. P. SHANMUGAPRIYA

SUPERVISOR

Professor

Department of Electronics &
Communication Engineering
Saranathan College of Engineering
Tiruchirappalli-620012.

Submitted for the project viva-voce examination held on_____

INTERNAL EXAMINER

EXTERNAL EXAMINER

CERTIFICATE OF EVALUATION

College Code : 8138

College Name : Saranathan College of Engineering

Branch : Electronics and Communication Engineering

Semester : VIII

Subject : EC3811 – Project Work

S.NO	Name of Students	Title of the Project	Name of the Supervisor with Designation
1	Nithish Kumar.G (813821106079)	Plant Disease Detection using yolo v8 Algorithm	Dr.P.SHANMUGAPRIYA PROFESSOR
2	Prashathan.V (813821106083)		
3	Rohith Joshua.M (813821106094)		
4	Subramanian.I (813821106116)		

The report of the project work is submitted by the above students in partial fulfilment for the award of Degree of Bachelor of Engineering in Electronics and Communication Engineering of Anna University: Chennai was confirmed to be the work done by the above students and evaluated on _____

INTERNAL EXAMINER

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

We would like to express our deep sense of gratitude to our secretary, **Shri S.RAVINDRAN**, Saranathan College of Engineering, for his sincere endeavor in educating us in our esteemed institution.

We thank our principal, **Dr.D.VALAVAN**, Saranathan College of Engineering, for giving permission to do this project. We sincerely thank our Dean (R&D), **Dr.R.NATARAJAN** Saranathan College of Engineering.

We are extremely thankful to our beloved head of the department, **Dr.M.SANTHI**, Department of Electronics and Communication Engineering, for her support and inspiration, also for the constant encouragement throughout the progress of the work. We are expressing our gratefulness to our beloved project coordinator. **Dr.M.PADMAA**, professor for coordinating the activities.

A special thanks to our panel members, **Dr.M.PADMAA**, Professor, **Dr.R.MUKESH**, Professor, **Dr.S.A.ARUNMOZHI**, Associate Professor, and **Dr.K.MALAISAMY**, Assistant Professor, for their valuable suggestions and interest at every stage of the project.

We are immensely pleased and deeply honored to extend our heartfelt thanks to our esteemed supervisor, **Dr.P.SHANMUGAPRIYA**, Professor, for gracefully accepting us as project students. The whole period of our project has been an excellent learning curve by virtue of her wisdom, advice, support, motivation, and valuable suggestions.

ABSTRACT

Plant diseases significantly impact global agricultural productivity, leading to economic losses and food security challenges. Traditional disease detection methods rely on manual inspection, which is time-consuming, error-prone, and requires expert knowledge. Recent advancements in deep learning have revolutionized plant disease detection, providing automated, efficient, and accurate solutions. This project utilizes YOLOv8 (You Only Look Once, Version 8), a state-of-the-art object detection model, to identify and classify plant diseases with high precision.

The proposed system involves collecting a diverse dataset of diseased and healthy plant images, preprocessing them for training, and fine-tuning YOLOv8 for optimized performance. The model is trained using Convolutional Neural Networks (CNNs) to detect various plant diseases such as leaf blight, rust, and powdery mildew. The implementation is carried out using Python, OpenCV, and PyTorch, with real-time detection enabled through edge devices such as Jetson Nano and Raspberry Pi.

Performance evaluation metrics, including mAP (Mean Average Precision), F1-score, and accuracy, are used to assess the model's effectiveness. Experimental results demonstrate a high detection accuracy, making the model suitable for real-world applications in smart farming and precision agriculture.

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	v
	LIST OF FIGURES	ix
	LIST OF TABLES	xi
	LIST OF ABBREVIATIONS	xii
1	INTRODUCTION	1
1.1	BACKGROUND AND MOTIVATION	1
1.2	IMPORTANCE OF PLANT DISEASE DETECTION	3
1.3	OVERVIEW OF DEEP LEARNING IN AGRICULTURE	4
1.4	INTRODUCTION TO YOLO V8 FOR OBJECT DETECTION	6
1.5	PROBLEM STATEMENT	7
1.6	OBJECTIVES OF THE STUDY	8
1.7	SCOPE OF THE PROJECT	10
1.8	FACTORS RESPONSIBLE FOR PLANT DISEASES	11
1.9	APPLICATIONS OF THE SYSTEM	13
1.10	ADVANTAGES	14
2	LITERATURE REVIEW	16
3	EXISTING SYSTEM	24

3.1	TRADITIONAL METHODS OF DISEASE DETECTION	24
3.2	LIMITATIONS OF EXISTING SYSTEM	25
4	PROPOSED SYSTEM	26
4.1	PROPOSED ARCHITECTURE	26
4.2	BLOCK DIAGRAM OF PROPOSED SYSTEM	28
4.3	SYSTEM DESIGN: OVERVIEW	30
4.4	CORE CONCEPT OF YOLO V8	32
4.5	WORKING PRINCIPLE OF YOLO V8	34
4.6	DATASET SELECTION AND PREPROCESSING	36
4.7	YOLO V8 MODEL TRAINING AND OPTIMIZATION	38
4.8	PERFORMANCE METRICS AND EVALUATION	41
4.9	DATA FLOW IN THE DETECTION SYSTEM	44
4.10	COMPARISON BETWEEN YOLO MODEL	46
5	SYSTEM IMPLEMENTATION	49
5.1	SOFTWARE REQUIREMENTS	49
5.2	HARDWARE REQUIREMENTS	51
5.3	DATASET PREPARATION AND AUGMENTATION	53
5.4	MODEL TRAINING AND OPTIMIZATION	58
5.5	MODEL VALIDATION AND TESTING	62

6	RESULTS AND DISCUSSION	64
6.1	CONFUSION MATRIX ANALYSIS	64
6.2	ACCURACY AND LOSS TRENDS	65
6.3	PRECISION-RECALL CURVE ANALYSIS	67
6.4	PRECISION CONFIDENCE CURVE	68
6.5	F1-SCORE	69
6.6	RECALL	70
6.7	TRAINING AND VALIDATION BATCH ANALYSIS	71
6.8	PRECISION, RECALL AND F1-SCORE	72
6.9	COMPARISON TABLE FOR PERFORMANCE ANALYSIS	73
6.10	COMPARATIVE ANALYSIS OF YOLO VERSIONS	74
6.11	MODEL EVALUATION RESULTS	74
7	CONCLUSION	75
8	APPENDICES	76
	REFERENCES	81

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE NO.
1.1	Overview of Plant Disease Detection	2
1.2	Disease Samples	12
4.1	Proposed Architecture of Detection System	28
4.2	Proposed System Block Diagram	29
4.3	Convolutional Neural Network Architecture	30
4.4	Yolo Architecture	31
4.5	Deep Learning Framework	32
4.6	Object Detection Framework	33
4.7	Data Preprocessing Steps	37
4.8	Dataset Samples and Labelling	38
4.9	Yolo v8 Training Process	40
5.1	Confusion Matrix Example	56
5.2	Label Distribution	57
5.3	Training Batch Samples	57
6.1	Confusion Matrix	64
6.2	Training vs Validation Loss	65
6.3	Model Performance	66

FIGURE NO.	TITLE	PAGE NO.
6.4	Precision-Recall Curve	67
6.5	Precision-Confidence Curve	68
6.6	F1-Score	69
6.7	Recall Curve	70
6.8	Training and Validation Batch	71
6.9	Precision, Recall and F1-Score	72
8.1	Display Sample Images from training set	77
8.2	Train the model	78
8.3	Evaluate the model	79
8.4	Test the model	80

LIST OF TABLES

TABLE NO.	TITLE	PAGE NO.
1	Performance Metrics Formulas	44
2	YOLO model comparison	48
3	Dataset	54
4	Performance Analysis	74
5	Comparative analysis of YOLO versions	74
6	Model evaluation results	74

LIST OF ABBREVIATIONS

Abbreviation	Description
AI	Artificial Intelligence
CNN	Convolutional Neural Network
YOLO	You Only Look Once
YOLOv8	Eighth version of the YOLO object detection model
ML	Machine Learning
DL	Deep Learning
IoU	Intersection over Union (used in object detection evaluation)
FPS	Frames Per Second
TP	True Positive
FP	False Positive
FN	False Negative
TN	True Negative
RGB	Red-Green-Blue (Colour Model)
mAP	Mean Average Precision (used in model evaluation)
F1-Score	Harmonic mean of Precision and Recall
ReLU	Rectified Linear Unit (Activation Function)
SGD	Stochastic Gradient Descent
Adam	Adaptive Moment Estimation
GPU	Graphics Processing Unit
API	Application Programming Interface
IoT	Internet of Things
CSV	Comma-Separated Values (Data Format)

CHAPTER 1

INTRODUCTION

1.1 BACKGROUND AND MOTIVATION

Tomatoes are among the most widely cultivated crops globally, playing a vital role in food production and the agricultural economy. Advancements in technology, particularly in artificial intelligence (AI) and deep learning, have paved the way for innovative solutions in agriculture. The integration of AI-based systems in plant disease detection has gained significant traction due to their ability to analyze large amounts of data quickly and accurately. Deep learning models, such as Convolutional Neural Networks (CNNs), have demonstrated remarkable success in image recognition and classification tasks, making them ideal for agricultural applications. These models can be trained on vast datasets containing images of healthy and diseased plant leaves, allowing them to learn patterns and differentiate between various disease types with high precision.

One of the most promising deep learning architectures for real-time object detection is the YOLO (You Only Look Once) model. Unlike traditional machine learning techniques that require multiple steps to detect objects, YOLO processes the entire image in a single pass, making it significantly faster and more efficient. YOLOv8, the latest version of the YOLO family, offers improved accuracy, enhanced feature extraction capabilities, and better real-time performance. These characteristics make it an excellent choice for implementing automated disease detection in tomato plants.

The motivation behind this project stems from the pressing need for a reliable and efficient system that can identify tomato leaf diseases early, thereby empowering farmers to take prompt action. By leveraging YOLOv8, this study aims to develop an automated detection system that can classify different types of tomato leaf diseases with high accuracy. Such a system will not only reduce the reliance on

manual inspection but also minimize crop losses by enabling early and precise disease diagnosis. Additionally, integrating this technology with mobile applications or agricultural drones can further enhance accessibility and scalability, making it easier for farmers to monitor their crops efficiently.

Implementing an AI-driven plant disease detection system has far-reaching implications for modern agriculture. It can lead to improved crop management, reduced dependency on chemical pesticides, and increased overall yield. By providing real-time insights, the system can support precision farming techniques, ensuring that only affected areas are treated, thereby reducing costs and environmental impact. Moreover, the adoption of such technology can bridge the gap between traditional farming practices and digital agriculture, fostering a more sustainable and productive agricultural ecosystem.

Figure 1.1 provides an overview of the plant disease detection process, illustrating the key steps involved. The system begins with the input image of a tomato leaf, followed by preprocessing steps such as resizing, augmentation, and noise removal. Feature extraction then takes place using deep learning techniques, and finally, classification is performed, yielding a prediction output with a confidence score.

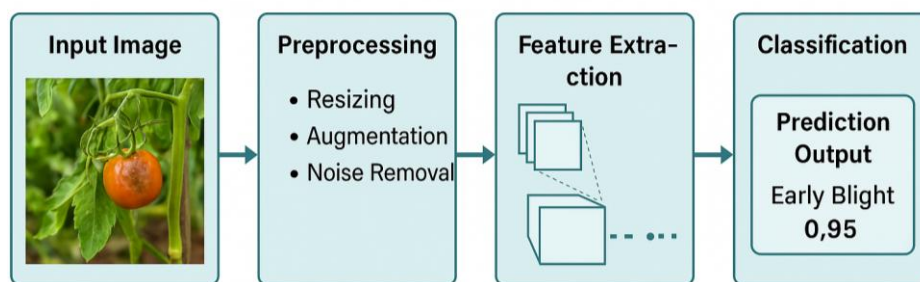


Figure 1.1 Overview of Plant Disease Detection

1.2 IMPORTANCE OF PLANT DISEASE DETECTION

Plant disease detection plays a crucial role in ensuring food security, maintaining crop quality, and minimizing economic losses in agriculture. Diseases can devastate crop yields, burden farmers financially, and disrupt food supply chains. Given the rising global population and growing demand for food, early and accurate detection is more essential than ever.

Diseases like bacterial spots, blight, and mosaic viruses can spread rapidly across farmlands. Early identification enables timely preventive action—such as applying fungicides or removing infected plants—reducing the chances of widespread outbreaks. Besides quantity, plant diseases also affect produce quality, often leading to reduced nutritional value, market rejection, and poor consumer satisfaction. Effective detection systems ensure only healthy crops reach the market, boosting both value and reputation.

Sustainable farming also hinges on efficient disease management. Traditional approaches often lead to overuse of chemical pesticides, which harm beneficial insects, pollute soil, and disrupt ecosystems. By adopting modern, AI-driven detection techniques like YOLO v8, farmers can target only affected areas, minimizing chemical use and promoting eco-friendly practices.

For farmers—especially smallholders—crop loss can be financially devastating. Many lack access to expert diagnosis or advanced tools. Automated systems reduce dependence on expert intervention and offer cost-effective, scalable solutions. Furthermore, these systems enhance food security by enabling consistent monitoring, improving yield reliability, and avoiding crises due to undetected infections.

Modern detection tools powered by deep learning and computer vision allow for real-time analysis of plant health. YOLO v8, for instance, can classify and localize diseases accurately using image datasets, enabling precise and immediate

response. These technologies also support agricultural research, helping scientists develop disease-resistant crops and innovative treatment strategies.

1.3 OVERVIEW OF DEEP LEARNING IN AGRICULTURE

Deep learning has revolutionized modern agriculture by introducing automation, precision, and intelligent decision-making into traditional farming practices. Once heavily reliant on manual methods and expert intuition, the agricultural sector now leverages artificial intelligence (AI) to improve productivity, optimize resource utilization, and ensure sustainable food production. As a subfield of machine learning, deep learning uses artificial neural networks to process vast and complex datasets. This allows for the detection of intricate patterns, accurate predictions, and data-driven insights. Its integration into agriculture addresses critical challenges such as climate variability, soil degradation, labour shortages, and the increasing global demand for food.

One of the most significant applications of deep learning in agriculture is plant disease detection. Convolutional Neural Networks (CNNs), a powerful deep learning architecture, are trained on large image datasets of crops to identify symptoms of various diseases with remarkable precision. These systems analyze leaf patterns, colour changes, and texture anomalies to detect infections at an early stage, enabling timely intervention. In addition to disease identification, CNNs can distinguish between healthy plants, weeds, and pests, making them highly effective in precision farming. This targeted approach supports environmentally sustainable practices by reducing the excessive and often unnecessary use of pesticides.

Deep learning also plays an important role in yield prediction. By analyzing historical climate patterns, current weather data, soil health metrics, and previous yield records, AI models can forecast expected harvest outputs. Such predictions help farmers plan their planting schedules, resource distribution, and marketing strategies. Soil analysis is another vital area where deep learning models assess

moisture content, nutrient levels, and pH balance, offering real-time suggestions for optimized fertilization and irrigation. This ensures that crops receive exactly what they need to thrive, improving both yield quality and quantity while conserving water and nutrients.

In the field of livestock management, deep learning is used to monitor the health and behaviour of animals. Smart surveillance systems powered by AI analyze movement, feeding patterns, and vital signs to detect early signs of disease or stress. This proactive approach enhances animal welfare and reduces economic losses due to undiagnosed illnesses. The integration of IoT devices and drones further enhances these capabilities, allowing farmers to monitor large fields and herds in real time. Smart cameras and sensors equipped with AI algorithms provide alerts on anomalies, improving operational efficiency and reducing the need for constant manual supervision.

The impact of deep learning extends beyond field-level analysis. Post-harvest operations such as crop sorting, grading, and packaging have also been automated using AI-driven robotic systems. These systems assess size, shape, ripeness, and defects, ensuring that only top-quality produce reaches the market. This reduces post-harvest losses, maximizes market value, and maintains food quality standards.

Moreover, sustainability is a key benefit of deep learning in agriculture. By enabling precision in every aspect of farming—from irrigation and pesticide use to harvesting—these technologies reduce environmental impact and help conserve natural resources. Farmers can minimize waste, avoid over-application of chemicals, and implement environmentally conscious farming methods with greater ease.

Looking ahead, the scope of deep learning in agriculture is poised to grow as AI models become more advanced and accessible. The fusion of deep learning with

robotics, satellite imagery, remote sensing, and cloud computing will unlock new possibilities in smart farming. Governments and agricultural organizations are also recognizing this potential, offering subsidies and incentives to promote the adoption of AI technologies in rural areas. With increased investment in agritech startups and research, deep learning is set to remain a central force in the advancement of global agriculture, helping farmers meet modern challenges and ensure food security for future generations.

1.4 INTRODUCTION TO YOLO V8 FOR OBJECT DETECTION

Object detection is a vital component of computer vision, enabling machines to identify, classify, and localize objects in images and videos. Among the leading models in this field is YOLO (You Only Look Once), with YOLO v8 being the most advanced and efficient version to date. Known for its real-time performance, high accuracy, and lightweight architecture, YOLO v8 is ideal for applications in agriculture, surveillance, medical imaging, and autonomous systems.

YOLO v8 uses a single-shot detection approach, processing the entire image in one pass and dividing it into a grid to detect multiple objects simultaneously. This significantly reduces inference time, making it highly effective for real-time agricultural tasks such as tomato leaf disease detection.

Key Features of YOLO v8

Improved architecture with advanced backbones like CSPDarknet and PAN, enhancing feature extraction.

Anchor-free detection, which simplifies training and increases accuracy by eliminating dependency on predefined anchor boxes.

Multiple model variants (Nano, Small, Medium, Large) cater to different computational requirements, offering flexibility across devices.

Optimized for edge devices, enabling deployment on drones, mobile phones, and embedded systems for real-time use.

Seamless integration with the Ultralytics Python framework, making training and deployment accessible even for non-experts.

YOLO v8 in Agriculture

In agricultural applications, YOLO v8 excels in plant disease detection. When trained on datasets of healthy and infected leaves, it can quickly identify diseases, allowing timely interventions to prevent crop loss. Combined with drones, YOLO v8 enables large-scale field monitoring, detecting pests, diseases, and nutrient deficiencies. It also supports automated harvesting, identifying ripe produce for robotic picking, and sorting post-harvest to increase supply chain efficiency.

Advantages

1. Real-time performance ensures fast decision-making in field conditions.
2. High accuracy in detecting small or occluded objects boosts reliability.
3. Low computational cost makes it suitable for general-purpose hardware.
4. Versatile implementation across agriculture, healthcare, surveillance, and more.
5. Ease of use through pre-trained models and user-friendly libraries.

1.5 PROBLEM STATEMENT

Agriculture remains the backbone of global food production, yet plant diseases continue to challenge productivity, quality, and sustainability. Traditional disease detection methods—primarily manual inspection—are slow, labour-intensive, and prone to inaccuracies due to symptom variability and environmental factors. In large-scale farms, relying solely on expert diagnosis becomes impractical. This gap highlights the urgent need for an automated, accurate, and real-time system that supports farmers with timely insights for disease management.

Tomatoes, one of the world's most widely cultivated crops, are especially vulnerable to various diseases such as early blight, late blight, and bacterial spot. These diseases reduce yield and quality while increasing dependency on pesticides, which negatively affect the environment and human health. Existing image processing and machine learning techniques often lack generalization capabilities, real-time performance, and scalability, making them less suitable for broad deployment in agriculture.

To overcome these limitations, this project proposes an automated disease detection system using YOLO v8, a cutting-edge deep learning model known for its speed, accuracy, and real-time detection capabilities. YOLO v8 offers an anchor-free architecture, efficient feature extraction, and compatibility with edge devices, making it ideal for detecting multiple tomato leaf diseases simultaneously in real-world conditions.

The system will involve training YOLO v8 on a diverse dataset of healthy and diseased tomato leaf images. A robust pipeline will be developed—from image preprocessing to real-time disease classification—supported by a user-friendly interface (mobile or web-based). This will empower farmers to monitor crop health, reduce reliance on chemical treatments, and take prompt action against disease spread.

Ultimately, the goal is to enhance agricultural productivity, enable sustainable farming practices, and provide accessible disease detection tools to farmers everywhere.

1.6 OBJECTIVES OF THE STUDY

The primary objective of this study is to develop an efficient, accurate, and real-time tomato leaf disease detection system using the YOLO v8 deep learning algorithm. Traditional manual inspection methods for identifying plant diseases are time-consuming and prone to human error. By leveraging artificial

intelligence, this study aims to create an automated, AI-powered system that minimizes human intervention while improving detection accuracy, contributing to sustainable and productive agriculture.

A key goal is to enhance the precision and speed of disease classification. YOLO v8 is selected for its superior performance in object detection tasks, particularly its real-time processing capability and improved feature extraction. The model will be fine-tuned with optimal hyperparameters and trained on a diverse image dataset using augmentation techniques to ensure it can generalize well in varied agricultural conditions. The study also focuses on deploying the model on low-power devices like smartphones and drones, making it practical for field-level applications.

Another objective is the preparation and preprocessing of a comprehensive dataset. High-quality images of both healthy and diseased tomato leaves will be collected, labelled, and augmented to simulate diverse environmental conditions and stages of disease progression. This ensures the model is robust against variability in lighting, background, and leaf appearance.

The study will also evaluate the system's performance using metrics such as precision, recall, F1-score, and mAP. A comparative analysis with other models like Faster R-CNN and SSD, as well as previous YOLO versions (v4, v5), will be conducted to validate YOLO v8's improvements in accuracy and computational efficiency.

To ensure real-world usability, the study aims to design a user-friendly interface—via a mobile app or web platform—enabling farmers to upload leaf images and receive instant results. Potential integration with IoT-based smart farming systems will also be explored for large-scale, automated crop monitoring.

Lastly, the research will assess the broader agricultural impact of the system, focusing on reduced pesticide use, lower operational costs, and improved crop

yield. By supporting early disease diagnosis and targeted treatment, the system aligns with sustainable farming practices and long-term productivity goals.

In summary, this study strives to deliver a scalable, accessible, and intelligent disease detection solution that transforms how tomato leaf diseases are identified and managed, benefitting farmers globally.

1.7 SCOPE OF THE PROJECT

This project focuses on developing an intelligent tomato leaf disease detection system using the YOLO v8 deep learning algorithm. It aims to overcome limitations of traditional disease detection methods by providing a real-time, high-accuracy, and automated solution. The system will classify tomato leaves as healthy or diseased and identify specific disease types such as early blight, late blight, leaf mold, and bacterial spots, ultimately supporting farmers, agronomists, and agricultural researchers in early diagnosis and crop management.

A core aspect of the project is dataset collection and preprocessing. The model will be trained on a diverse dataset of tomato leaf images gathered from both public databases and real-world agricultural settings. Images will undergo normalization, augmentation, and noise reduction to enhance the model's generalization and robustness under varied environmental conditions.

The project also includes the implementation, training, and optimization of the YOLO v8 model. Advanced hyperparameter tuning will be applied to improve accuracy while minimizing false detections. The system's performance will be compared to earlier YOLO versions (v4, v5) and other detection models like Faster R-CNN and SSD to evaluate speed, accuracy, and efficiency for agricultural use cases.

Deployment is another major focus. The model will be adapted for multiple platforms, including web apps, mobile devices, and edge devices like Raspberry Pi and Jetson Nano, making it practical for use in remote or rural areas.

Additionally, integration with UAVs (drones) will be explored for large-scale crop monitoring and automated field scanning.

The study will assess the economic and environmental benefits, such as reduced pesticide usage, lower operational costs, and improved yields. The system will be tested across different field conditions, and its usability will be validated with feedback from farmers and agricultural professionals.

Finally, the project is designed with scalability in mind. It can be extended to detect diseases in other crops, integrated with IoT sensors for real-time farm monitoring, and improved with transfer learning techniques and multilingual support, ensuring wide adaptability in future agricultural applications.

1.8 FACTORS RESPONSIBLE FOR LEAF DISEASE

Tomato plants are highly vulnerable to various diseases caused by biotic and abiotic factors, significantly impacting growth, yield, and crop health. These diseases contribute to economic losses and food insecurity. Effective management begins with understanding the underlying causes, including environmental, biological, and agricultural factors.

One major cause is pathogenic microorganisms—bacteria, fungi, and viruses. These pathogens enter through contaminated soil, air, water, or insect vectors. For example, *Xanthomonas campestris* causes bacterial spot, while fungal infections like early blight (*Alternaria solani*) and late blight (*Phytophthora infestans*) thrive in humid conditions, leading to lesions, yellowing, and defoliation. Viral infections such as Tomato Yellow Leaf Curl Virus (TYLCV) cause curled, yellow leaves and stunted growth as shown in Figure 1.2.

Environmental factors like high humidity and prolonged wet leaf surfaces promote disease development. Rain splashes spread spores, and warm, damp conditions increase the risk of fungal diseases such as leaf mold. Temperature swings can weaken plant immunity, making them more prone to infection.

Soil quality and nutrient deficiencies also play a key role. Poor drainage can lead to root rot, while imbalanced nutrients—especially nitrogen, potassium, and calcium—affect leaf resilience. Low nitrogen weakens foliage; excess nitrogen creates dense canopies that trap moisture. Potassium deficiency increases vulnerability to Tomato Mosaic Virus (TMV), resulting in mosaic-like discoloration.

Insect pests like whiteflies, aphids, and thrips act as vectors for viral infections. Whiteflies, for example, transmit TYLCV. Spider mites (*Tetranychus spp.*) feed on leaf sap, causing bronzing and reducing disease resistance.

Poor agricultural practices further amplify disease risk. Dense planting reduces airflow, encouraging humidity build-up. Overhead irrigation wets leaves, promoting fungal spread. Infected seeds or seedlings can initiate outbreaks, and lack of crop rotation allows soil-borne pathogens to persist.

As illustrated in Figure 1.2, diseases affect tomato plants in diverse ways. Addressing these factors through integrated strategies—such as pest control, better irrigation, resistant varieties, and crop rotation—can significantly reduce disease impact and improve plant health.

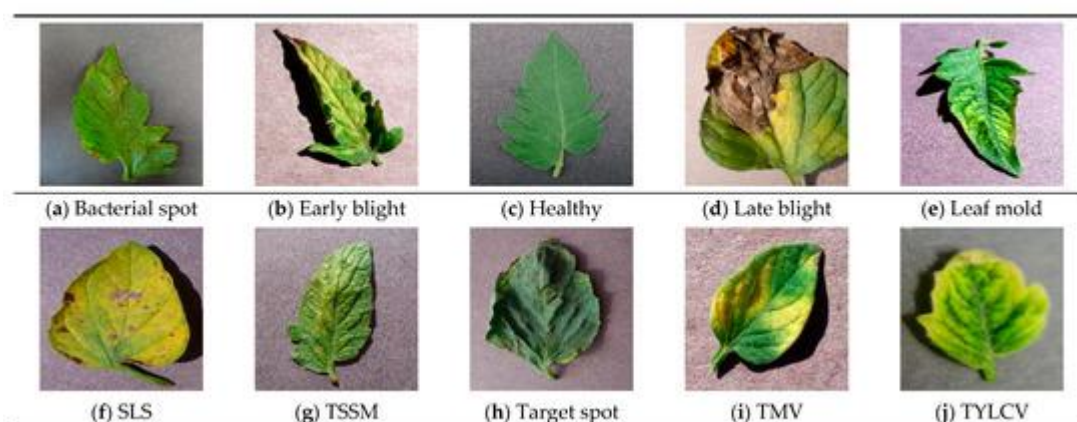


Figure 1.2 Disease Samples

1.9 APPLICATION OF THE SYSTEM

The YOLO v8-based tomato leaf disease detection system offers significant advantages for modern agriculture, improving disease identification, crop management, and productivity. Traditional manual observation methods are slow, labour-intensive, and prone to errors, whereas this AI-driven system provides real-time, precise disease detection, allowing for timely interventions and better decision-making. Early disease detection reduces crop losses, improves yield quality, and supports sustainable farming practices.

Key applications include real-time field monitoring using drones, fixed cameras, or handheld devices to detect diseases in their early stages. This reduces the need for excessive pesticide use and allows farmers to receive automated alerts, ensuring prompt action. In precision agriculture, the system enables targeted treatment, minimizing chemical usage and environmental impact while optimizing resource utilization. It also aids in disease progression tracking, crop rotation planning, and preventive strategies.

In agricultural research and education, the system supports disease trend analysis, environmental factor studies, and the development of disease-resistant tomato varieties. It offers practical learning opportunities for students and professionals, promoting AI-driven plant disease detection and smart farming solutions. Additionally, integrating the system into mobile apps allows farmers in remote areas to capture images of tomato leaves, receive diagnoses, and access treatment recommendations, reducing reliance on agricultural consultants.

The system is also invaluable in greenhouse farming, where it detects early disease symptoms and prevents outbreaks in tightly spaced plants. Automated spraying mechanisms ensure efficient treatment application, reducing chemical exposure. In agribusiness, automated disease detection during sorting and

packaging ensures high-quality, disease-free tomatoes, reducing waste and meeting international quality standards, boosting profitability.

The system supports predictive analytics for policymakers, enabling disease outbreak tracking, risk prediction, and region-specific control measures. It enhances resource allocation, supports targeted advisory services, and trains extension officers with advanced tools for disease identification and management. Overall, the YOLO v8-based system revolutionizes agricultural practices, contributing to sustainable farming and improving food security.

1.10 ADVANTAGES

The integration of YOLO v8-based tomato leaf disease detection into agriculture offers numerous advantages, transforming disease management into a more efficient, accurate, and scalable process. By utilizing deep learning and computer vision, the system enables early disease detection, reduces reliance on manual inspections, optimizes resource usage, and boosts agricultural productivity. It benefits farmers, researchers, agribusinesses, and policymakers, making it an essential tool for modern agriculture.

A major advantage of this system is its high-speed, real-time disease detection. Unlike traditional methods, which rely on manual inspections, YOLO v8 instantly identifies diseases through images captured by drones, cameras, or mobile devices. This allows for immediate action, preventing the spread of diseases, especially in large-scale farms where manual monitoring is impractical. Additionally, the system provides high accuracy and precision, ensuring the correct classification of diseases like early and late blight, minimizing misdiagnosis and unnecessary pesticide use.

The system also reduces labour costs and dependency on experts. Traditional disease detection requires skilled agronomists or experienced farmers, but with YOLO v8, any farmer can diagnose diseases simply by capturing an image. This

makes advanced technology accessible to small-scale farmers, especially in rural or developing areas, where expert help may be scarce. Moreover, it promotes sustainable farming by reducing pesticide overuse. Early detection allows for targeted pesticide application, minimizing environmental harm and supporting eco-friendly practices.

Scalability and adaptability are additional benefits. The system can be deployed in various farming environments, from open fields to greenhouses and hydroponic systems. It can also be expanded to monitor other crops, providing a comprehensive plant health monitoring solution. Furthermore, it aids decision-making through predictive analytics, helping farmers track disease trends, optimize planting schedules, and improve crop management strategies. Aggregated data can also guide policymakers in making informed decisions on crop protection and disease control.

Finally, the integration with mobile applications makes the system accessible and user-friendly. Farmers can use smartphones to capture images of affected leaves, receive instant diagnoses, and get treatment recommendations. This mobile accessibility is especially valuable for remote and small-scale farmers who lack access to advanced agricultural research, offering an end-to-end solution for disease management.

CHAPTER 2

LITERATURE REVIEW

2.1 TITLE: A Novel Deep Learning Method for Detection and Classification of Plant Diseases

AUTHORS: W. Albattah, M. Nawaz, A. Javed, M. Masood, and S. Albahli

YEAR: 2022

DESCRIPTION:

This study introduces a novel deep learning-based approach for detecting and classifying plant diseases, addressing the limitations of traditional manual inspection and early machine learning models. The authors propose a model that utilizes Convolutional Neural Networks (CNNs), optimized through advanced feature extraction and training strategies to improve the accuracy of disease classification in plant leaves.

The methodology involves the preprocessing of leaf images followed by automatic feature learning through CNN layers. The proposed model is evaluated on a benchmark plant disease dataset and achieves high accuracy in both detection and classification tasks. The study also explores the impact of various hyperparameters on model performance and discusses the robustness of the model under real-world conditions, such as variations in lighting and leaf orientation.

Through extensive experiments, the authors demonstrate that their method significantly outperforms traditional machine learning approaches and offers greater scalability and automation, making it suitable for large-scale agricultural applications. The findings highlight the potential of deep learning techniques in advancing precision agriculture and enabling real-time disease monitoring systems for farmers.

2.2 TITLE: Classification of Cotton Leaf Diseases Using Transfer Learning—DenseNet-121

AUTHORS: Arathi, B., and Dulhare, U. N.

YEAR: 2023

DESCRIPTION:

This research presents a transfer learning-based approach for the classification of cotton leaf diseases using the DenseNet-121 deep learning model. The study addresses the challenge of recognizing and categorizing plant diseases with high accuracy in varying environmental conditions and image complexities.

The authors utilize the DenseNet-121 architecture, pre-trained on large image datasets, and fine-tune it using a specialized cotton leaf dataset that includes images of diseased and healthy samples. The model is trained to classify different disease types such as bacterial blight and curl virus, showing significant improvements over traditional CNNs and earlier deep learning models.

The system achieves a high level of classification accuracy and generalization by leveraging DenseNet's unique feature reuse and efficient connectivity structure. This study highlights the effectiveness of transfer learning in agriculture, especially in cases where labelled data is limited. It demonstrates that fine-tuning deep pretrained networks can produce robust results for real-world disease detection applications in precision farming. Moreover, the transfer learning approach reduces training time and computational cost, making it a practical solution for deploying disease detection models in field conditions. The study also emphasizes the importance of data augmentation techniques in improving the model's robustness against overfitting and ensuring consistent performance across diverse environments. By applying DenseNet-121 to plant disease classification, this research contributes to the development of smarter, more efficient tools for sustainable agriculture.

2.3 TITLE: Tomato Leaf Disease Classification via Compact CNNs with Transfer Learning and Feature Selection

AUTHOR: O. Attallah

YEAR: 2023

DESCRIPTION:

This paper presents a method for tomato leaf disease classification using compact convolutional neural networks (CNNs) augmented with transfer learning and feature selection techniques. The aim of the study is to improve the accuracy of disease detection while reducing the computational load of traditional CNN models, making them more suitable for resource-constrained environments like mobile devices. The model utilizes pre-trained CNNs through transfer learning, which allows the system to benefit from the knowledge learned from large datasets, even with limited data available for tomato leaves. Feature selection techniques are employed to identify the most relevant features, ensuring the model focuses on the most informative aspects of the images, thus enhancing classification performance. The method is evaluated on a comprehensive dataset of tomato leaf images, achieving high accuracy and outperforming conventional deep learning approaches. The results indicate that this hybrid approach can significantly improve both the efficiency and effectiveness of plant disease detection systems, providing a valuable tool for precision agriculture. By providing a more efficient and accurate tool, this study contributes to the advancement of intelligent agriculture systems, promoting sustainable farming practices. The method's potential for early disease detection could also help reduce the use of pesticides, minimizing environmental impact. Furthermore, the approach offers scalability, making it adaptable to a variety of crop species beyond tomatoes, which could lead to broader applications in the agricultural industry.

2.4 TITLE: Deep Learning Models for Cotton Leaf Disease Detection with VGG-16

AUTHORS: Bin Naeem, A., Senapati, B., Chauhan, A. S., Kumar, S., Gavilan, J. C. O., and AbdelRehim, W. M.

YEAR: 2023

DESCRIPTION:

This study investigates the application of deep learning models, particularly the VGG-16 architecture, for detecting diseases in cotton leaves. The authors utilize VGG-16, a deep convolutional neural network pre-trained on large datasets, and adapt it to classify various cotton leaf diseases. By employing transfer learning, the model is fine-tuned on a specialized cotton leaf disease dataset, allowing it to achieve high accuracy even with limited data. The research emphasizes the importance of image preprocessing techniques such as augmentation and normalization to enhance model performance. The results show that VGG-16 can effectively identify diseases like bacterial blight and cotton leaf curl, offering a reliable tool for automated disease detection in cotton crops. This work highlights the potential of deep learning models in advancing agricultural practices, improving disease management, and promoting sustainable farming techniques. The authors also explore the possibility of extending this model to other crops, offering a scalable solution for global agricultural systems. By optimizing deep learning models for real-world deployment, the research opens avenues for precision farming technologies that could significantly reduce crop loss and pesticide use. The findings underscore the effectiveness of VGG-16 as a tool for early detection, enabling farmers to take timely actions and mitigate the impact of diseases on cotton yields.

2.5 TITLE: Grape Leaf Disease Identification with Sparse Data via GANs and CNNs

AUTHORS: Chen, Y., and Wu, Q.

YEAR: 2023

DESCRIPTION:

This paper introduces a hybrid approach for grape leaf disease identification that combines Generative Adversarial Networks (GANs) and Convolutional Neural Networks (CNNs) to tackle the challenge of sparse data. The authors use GANs to generate synthetic images of grape leaves affected by various diseases, thus augmenting the limited available dataset. These synthetic images are then utilized to train a CNN model for accurate disease classification. The focus is on diseases such as powdery mildew and downy mildew, which are commonly found in grape crops. The study shows that the GAN-generated images significantly improve the performance of the CNN, allowing the model to effectively identify grape leaf diseases despite having a small dataset. This work demonstrates the potential of combining GANs with CNNs to enhance agricultural disease detection systems, even when data is scarce. The approach also minimizes the risk of overfitting, a common issue when training deep learning models on small datasets. Additionally, the method's scalability opens up opportunities for expanding it to other crops and diseases. By integrating these advanced techniques, the study provides a promising framework for building more robust plant disease detection systems. The success of the hybrid approach could lead to better disease management strategies, improving crop yields and reducing the need for chemical interventions.

2.6 TITLE: Plant Disease Recognition Model Based on Improved YOLOv5

AUTHORS: Chen, Z., Ruhui, W., Yiyang, L., Chuyang, L., Siyu, C., Zhineng, Y.

YEAR: 2022

DESCRIPTION:

This paper proposes an improved version of the YOLOv5 model for plant disease recognition. The authors enhance the original YOLOv5 architecture by integrating advanced techniques such as attention mechanisms and multi-scale feature extraction to improve its ability to detect and classify plant diseases more accurately. The modified YOLOv5 model is trained on a large-scale plant disease dataset, which includes a wide variety of crops and diseases. By refining the feature extraction process, the model achieves better detection performance and reduces false positives. The study demonstrates that the improved YOLOv5 not only excels in detecting diseases in plant leaves but also provides faster inference times, making it suitable for real-time agricultural applications. The results show that this enhanced model significantly outperforms traditional methods, highlighting its potential for large-scale deployment in precision agriculture for disease management. The model's ability to generalize across different plant species and disease types showcases its robustness and adaptability. The introduction of attention mechanisms allows the model to focus on the most relevant features of the input image, leading to a reduction in computational cost while maintaining high accuracy. The authors also highlight the potential for this approach to be integrated into automated agricultural systems, offering timely and cost-effective disease detection solutions for farmers. This research sets the foundation for further developments in plant health monitoring, contributing to sustainable farming practices and improved crop yield.

2.7 TITLE: Apple Leaf Disease Identification Using Genetic Algorithm and Feature Selection

AUTHORS: Chuanlei, Z., Shanwen, Z., Jucheng, Y., Yancui, S., and Jia, C.

YEAR: 2017

DESCRIPTION:

This paper presents an innovative approach to apple leaf disease identification by combining Genetic Algorithms (GA) with feature selection techniques. The authors propose a method to improve the accuracy of disease classification by first selecting the most relevant features from the apple leaf images and then applying a genetic algorithm to optimize the feature selection process. The model is trained on a dataset consisting of various diseases affecting apple leaves, such as apple scab and powdery mildew. The GA helps in selecting a subset of features that are most discriminative, thus improving the efficiency and accuracy of the disease detection model. The results show that the genetic algorithm-based feature selection significantly reduces the dimensionality of the data while maintaining or even enhancing classification accuracy. This approach provides a robust solution for apple leaf disease identification, enabling early detection and management of plant health issues. By focusing on the most relevant features, the method reduces computational complexity, making it more suitable for real-time applications. The combination of GA and feature selection can be easily adapted to other crops, showing its versatility in agricultural disease detection. The study also highlights the potential for integrating this approach into precision farming systems to support proactive disease management strategies. The authors emphasize the effectiveness of their model in environments with limited computational resources. The proposed system can be easily integrated into existing agricultural frameworks, allowing for practical, real-time disease detection.

2.8 TITLE: Detection of Leaf Disease Using PCA and Linear SVM

AUTHORS: Dhinesh, E., Jagan, A., et al.

YEAR: 2019

DESCRIPTION:

This paper proposes a method for detecting plant leaf diseases using Principal Component Analysis (PCA) for dimensionality reduction and a Linear Support Vector Machine (SVM) for classification. The approach aims to address the challenge of high-dimensional data in plant disease detection by reducing the feature space through PCA, which helps retain the most significant characteristics of the leaf images. After dimensionality reduction, the reduced features are fed into a Linear SVM model to classify healthy and diseased leaves. The authors demonstrate that this method offers a significant improvement in both accuracy and computational efficiency when compared to traditional image classification methods. The study is particularly focused on diseases affecting a variety of crops, providing an efficient solution for early-stage disease detection in agricultural practices. The proposed technique shows promise for practical implementation in precision agriculture, particularly in environments with limited computational resources. The combination of PCA and Linear SVM ensures that the system can be deployed on low-power devices, making it suitable for real-time disease monitoring in the field. The method also shows high adaptability to various crops, which expands its potential for large-scale agricultural applications. This approach could lead to more cost-effective and sustainable farming practices by providing timely and accurate disease detection with minimal infrastructure requirements.

CHAPTER 3

EXISTING SYSTEM

3.1 TRADITIONAL METHODS OF DISEASE DETECTION

Traditionally, plant disease detection has been carried out through manual inspection by experienced farmers, agronomists, or agricultural scientists. These inspections typically involve visually analyzing the colour, texture, and shape of leaves, stems, and fruits to identify symptoms of various diseases. In the case of tomato plants, common indicators such as yellowing leaves, dark spots, curling, and mold formations are used as clues for diagnosing diseases like early blight, late blight, and bacterial spot.

Other conventional techniques include the use of laboratory tests such as polymerase chain reaction (PCR), enzyme-linked immunosorbent assay (ELISA), and microscopic examinations. While these methods are relatively accurate, they are also time-consuming, require skilled personnel, and are often inaccessible to farmers in rural areas due to cost and logistical challenges.

In recent years, digital approaches such as image processing and basic machine learning models have also been explored. These include algorithms based on k-nearest neighbours (KNN), support vector machines (SVM), and decision trees applied to handcrafted features like colour histograms or texture patterns. However, these early-stage models have limited ability to deal with complex variations in real-world images such as lighting, occlusion, or background clutter.

Although these approaches laid the groundwork for automated detection systems, they fall short in terms of scalability, speed, and consistency. Their reliance on manual feature extraction and expert intervention makes them less suitable for real-time or large-scale agricultural monitoring.

3.2 LIMITATIONS OF EXISTING SYSTEM

Despite being the foundation of plant disease detection, existing systems have several inherent limitations that restrict their effectiveness, especially in real-time and large-scale applications.

One of the key drawbacks is lack of automation. Manual inspection is subjective, inconsistent, and inefficient when dealing with large farms. The accuracy of detection depends heavily on the expertise of the person performing the inspection, which introduces variability and increases the risk of misdiagnosis.

Another limitation is the inability to process large volumes of data quickly. Traditional systems are not equipped to handle high-resolution images or continuous monitoring, which are essential in modern agriculture. This bottleneck makes early disease detection challenging, allowing infections to spread unchecked.

Limited scalability and accessibility also pose challenges. Laboratory-based methods, while accurate, are expensive and slow, making them impractical for farmers in developing regions. Similarly, classical image processing techniques require significant calibration for different conditions and often fail when tested on new, unstructured datasets.

Furthermore, earlier machine learning methods lack deep feature extraction capabilities. Unlike deep learning, these models require manual identification of relevant features, which is time-intensive and less accurate. They also struggle with variations in leaf shape, size, orientation, and lighting—factors that are common in real-world agricultural settings.

The limitations of traditional and early machine learning approaches highlight the need for advanced, real-time, and intelligent disease detection systems. These challenges set the stage for deep learning-based solutions such as YOLOv8, which offer superior accuracy, scalability, and adaptability.

CHAPTER 4

PROPOSED SYSTEM

4.1 PROPOSED SYSTEM ARCHITECTURE

The proposed system architecture for tomato leaf disease detection is designed to ensure efficient and accurate classification of diseased and healthy leaves. The system integrates image preprocessing, YOLOv8-based object detection, and a post-processing module to enhance detection accuracy. The architecture is illustrated in Figure 4.1, which provides an overview of the workflow from image input to final prediction and decision-making.

Input Layer

The input to the system consists of tomato leaf images captured using cameras or obtained from publicly available datasets. These images may contain healthy or diseased leaves, and the goal of the model is to accurately classify and localize affected areas. Since raw images may have varying resolutions and lighting conditions, preprocessing is applied before feeding them into the model.

Preprocessing Module

The preprocessing step is crucial for standardizing input data and enhancing the quality of features extracted by the deep learning model. This stage involves resizing images to a fixed dimension, normalizing pixel values, and applying data augmentation techniques such as rotation, flipping, and contrast adjustment. These techniques help improve the model's generalization ability and robustness against variations in real-world conditions.

YOLOv8 Model

The YOLOv8 algorithm serves as the core detection model. It is a state-of-the-art object detection framework known for its speed and accuracy. The model takes the pre-processed images as input and performs real-time detection of tomato leaf

diseases. YOLOv8 utilizes convolutional neural networks (CNNs) to extract hierarchical features from images and predict bounding boxes, class labels, and confidence scores for detected regions. The model is trained on a labelled dataset, enabling it to recognize multiple types of tomato leaf diseases effectively.

Prediction Output

Once the YOLOv8 model processes an image, it generates prediction outputs that include:

Bounding boxes: Indicating the detected diseased regions.

Class labels: Identifying the type of disease (e.g., early blight, late blight, leaf mold).

Confidence scores: Representing the model's certainty about each prediction.

This output is then forwarded to the post-processing module for further refinement and decision-making.

Post-processing & Decision Module

The post-processing module refines the model's predictions by filtering out low-confidence detections using a threshold value. Non-maximum suppression (NMS) is applied to remove duplicate bounding boxes and retain the most relevant detections. The final output is presented to the user, providing insights into the detected disease along with confidence levels. Based on the results, farmers and agricultural experts can take necessary actions to prevent disease spread and improve crop health.

By integrating these components, the proposed system ensures efficient, real-time, and accurate detection of tomato leaf diseases. The architecture shown in Figure 4.1 demonstrates the end-to-end workflow of the detection system, highlighting how YOLOv8 is used for disease classification and localization.

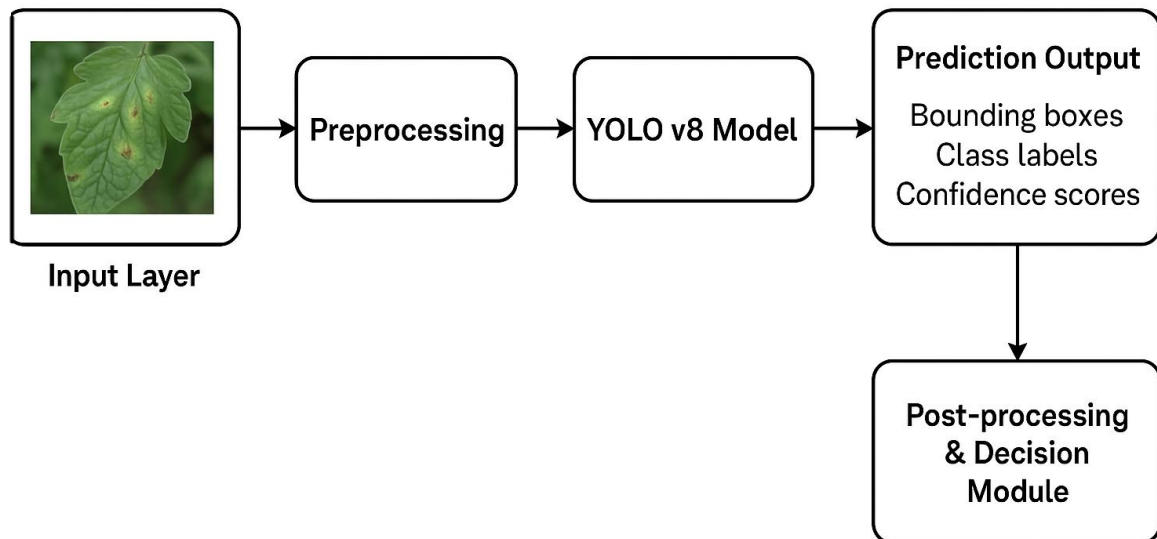


Figure 4.1 Proposed Architecture of the Detection System

4.2 BLOCK DIAGRAM OF PROPOSED SYSTEM

The system block diagram as shown in Figure 4.2 outlines the end-to-end workflow of the proposed tomato leaf disease detection system using the YOLOv8 model, ensuring accurate and real-time classification.

Input – Plant Leaf Images:

Images of healthy and diseased tomato leaves are collected using cameras or sourced from datasets. These serve as input for model training and testing.

Preprocessing:

Preprocessing improves image quality and model readiness:

Normalization: Standardizes pixel values.

Resizing: Scales images to a fixed input size.

Augmentation: Applies transformations (e.g., rotation, flipping) to improve model robustness.

YOLOv8 Detection Model:

Images are passed into the YOLOv8 model, which detects and classifies diseases in real time. The output includes:

1. Bounding boxes
2. Disease class labels
3. Confidence scores

Post-Processing:

Non-Maximum Suppression (NMS) is applied to remove redundant detections and retain the most accurate results.

Identification Results:

Final output displays disease type with confidence levels, helping users take timely action for disease management.

This streamlined system supports efficient, field-ready disease detection for smart agriculture.

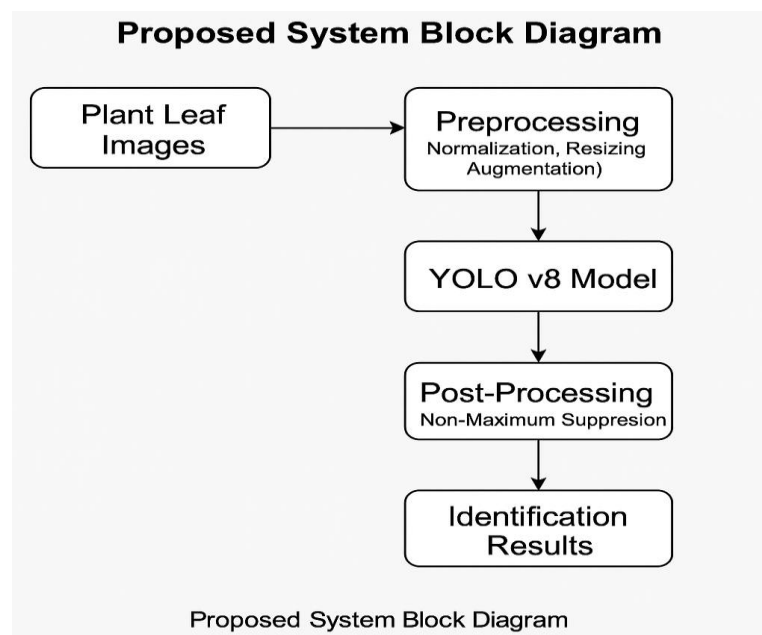


Figure 4.2 Proposed System Block Diagram

4.3 SYSTEM DESIGN: OVERVIEW

Convolutional Neural Networks (CNNs)

Convolutional Neural Networks (CNNs) form the backbone of the tomato leaf disease detection system, enabling effective feature extraction from input images. As shown in Figure 4.3, CNNs utilize convolutional layers to detect patterns such as edges, textures, and colour variations that distinguish healthy and diseased leaves. Pooling layers reduce the dimensionality of the extracted features while retaining important information, ensuring computational efficiency. Fully connected layers further process the extracted features, classifying them into different disease categories. CNNs play a crucial role in identifying disease-specific patterns and enhancing the model's overall accuracy.

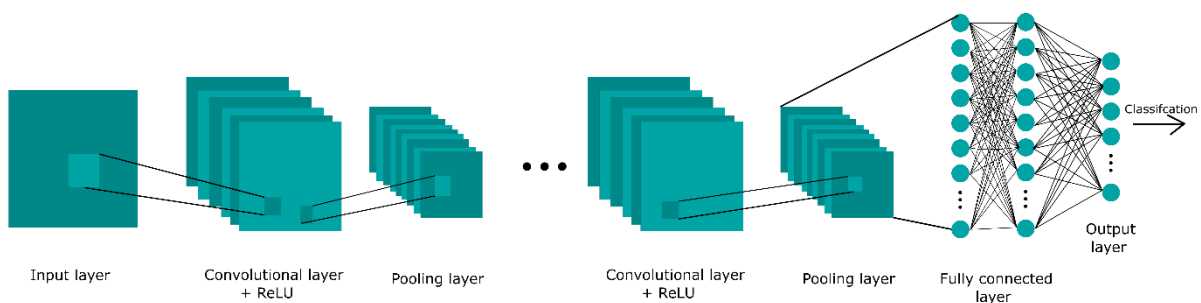


Figure 4.3 Convolutional Neural Networks Architecture

YOLO (You Only Look Once) Algorithm

The YOLOv8 model is employed for real-time object detection, making it ideal for agricultural applications. As illustrated in Figure 4.4, YOLOv8 processes an entire image in a single pass, ensuring high-speed detection while maintaining accuracy. Unlike traditional object detection models that analyze images in multiple steps, YOLO detects diseased regions instantly by applying bounding boxes with confidence scores. This capability allows for the rapid identification of plant diseases, making it an efficient solution for large-scale agricultural

monitoring. Additionally, YOLOv8's improved architecture, which includes better feature extraction and prediction mechanisms, enhances detection performance even under challenging conditions such as varying lighting and occlusions.

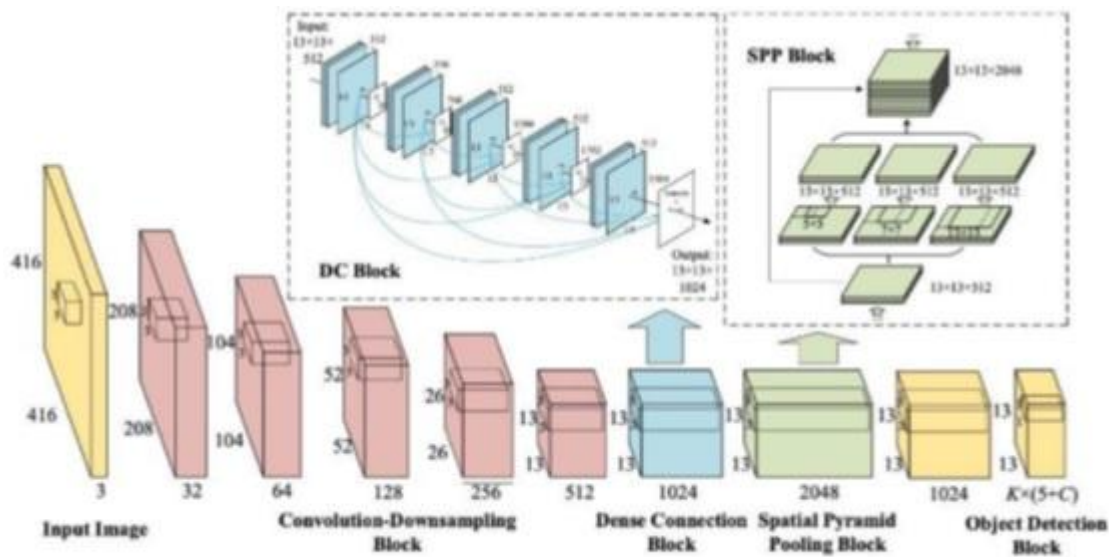


Figure 4.4 Yolo Architecture

Deep Learning Framework

The deep learning framework integrates CNNs and YOLOv8 into a structured pipeline for robust disease detection, as depicted in Figure 4.5. The system consists of several processing blocks, including convolutional and dense connection layers, which enhance feature learning and enable the model to detect diseases across different scales and orientations. The Spatial Pyramid Pooling (SPP) block further improves multi-scale feature extraction, ensuring that the model captures disease characteristics from various perspectives. The final detection head, powered by YOLOv8, classifies the diseased leaf regions with high precision. By combining CNNs, YOLO, and advanced deep learning techniques, the system provides an automated, scalable, and efficient approach

for identifying tomato leaf diseases, contributing to precision agriculture and early disease management.

Deep Learning Pipeline

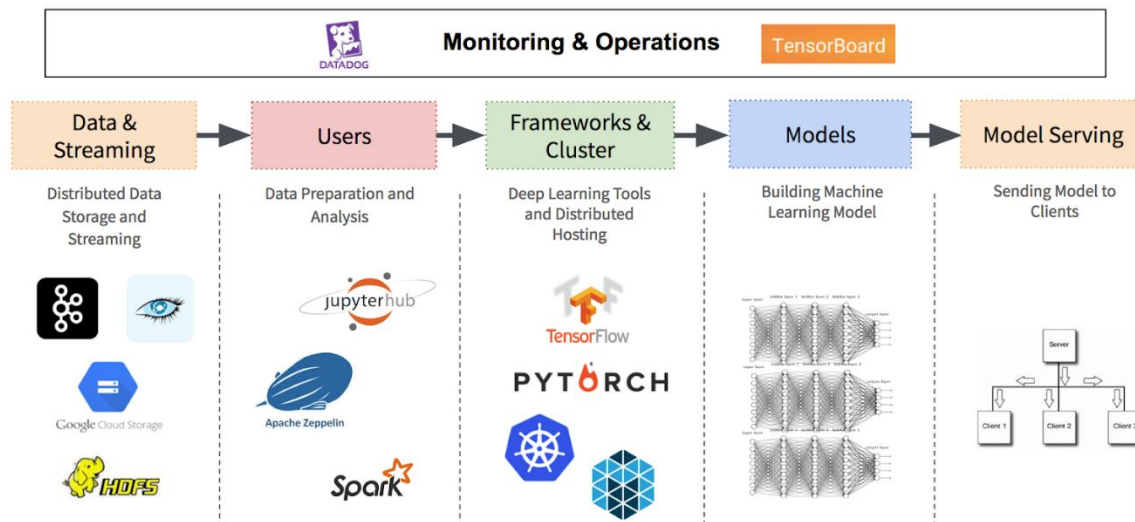


Figure 4.5 Deep Learning Framework

4.4 CORE CONCEPT OF YOLO V8

YOLOv8 (You Only Look Once version 8) is a cutting-edge object detection algorithm that has gained attention for its speed and accuracy in real-time applications. In the context of plant disease detection, particularly for tomato leaves.

At its core, YOLOv8 is a one-stage detector, meaning it performs object localization and classification in a single forward pass through the neural network. This approach significantly reduces computation time compared to two-stage detectors like Faster R-CNN, making it ideal for real-time agricultural applications. YOLOv8 processes the entire image at once, dividing it into a grid and predicting bounding boxes and class probabilities for each cell.

The model architecture in YOLOv8 is built with improvements in backbone and head designs. It utilizes a CSPDarknet-inspired backbone for feature extraction, PAN-FPN for feature fusion, and a decoupled head that separates object classification and localization tasks. These enhancements improve the model's ability to detect small and subtle features, such as disease symptoms on leaves. The complete object detection pipeline is illustrated in Figure 4.6, which outlines how input images are processed through the YOLOv8 architecture to yield disease detection results.

In plant disease detection, YOLOv8 is trained on annotated images of healthy and diseased tomato leaves. During training, the model learns to associate specific visual patterns—like spots, discoloration, curling, or mold—with corresponding disease classes such as early blight, late blight, or TYLCV. Once trained, YOLOv8 can accurately detect the diseased region in a leaf, classify the type of disease, and assign a confidence score to the prediction.

The advantages of using YOLOv8 include high-speed inference, scalability to mobile and edge devices, and robust performance under varied lighting and background conditions. Its capability to generalize well on augmented and real-world data makes it suitable for deployment in agricultural fields, mobile apps, or drone-based surveillance systems.

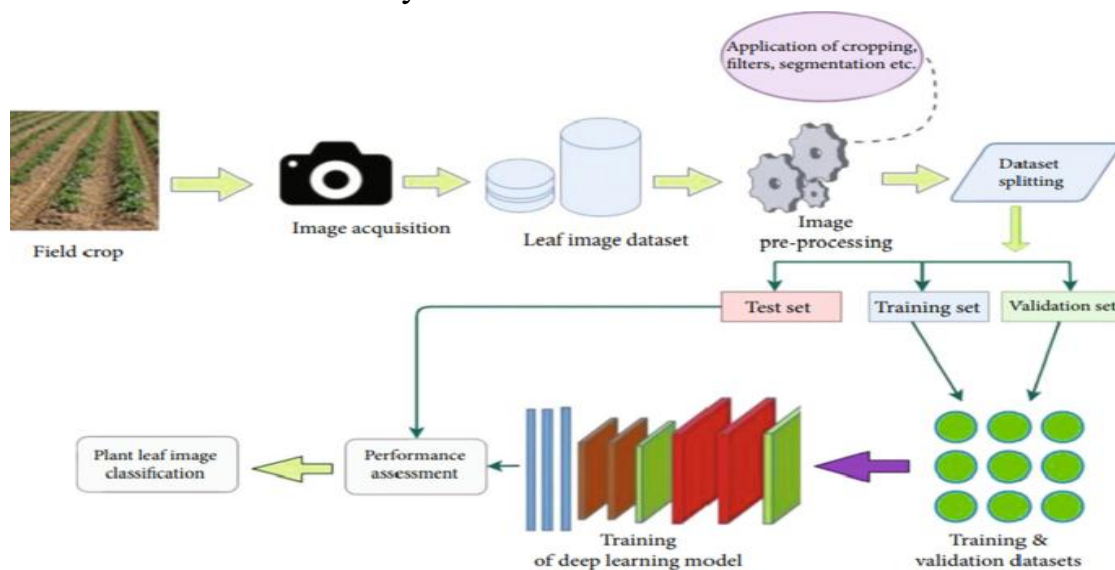


Figure 4.6 Object Detection Framework

4.5 WORKING PRINCIPLE OF YOLO V8

Image Input and Preprocessing

The working of YOLOv8 begins with an input image of a tomato leaf, which undergoes preprocessing to enhance detection accuracy. This involves resizing the image to match the model's input dimensions, normalizing pixel values to ensure consistency, and applying data augmentation techniques such as rotation, flipping, and contrast adjustment. These preprocessing steps help improve the model's robustness by enabling it to recognize diseased regions under various environmental conditions.

Feature Extraction through CNN Backbone

Once the pre-processed image is fed into YOLOv8, it passes through a deep convolutional neural network (CNN) backbone. This backbone extracts essential features such as texture, shape, and colour variations that distinguish healthy leaves from diseased ones. The backbone consists of multiple convolutional layers, each responsible for detecting progressively complex patterns. Unlike earlier YOLO versions, YOLOv8 incorporates an improved feature extraction network that enhances spatial awareness and context understanding, allowing it to detect diseases more accurately.

Object Localization and Bounding Box Regression

After feature extraction, YOLOv8 predicts bounding boxes that localize diseased areas on the tomato leaf. This step involves a process known as regression, where the model assigns coordinates (x, y, width, height) to each detected object. Unlike region-based approaches that first generate potential object proposals before classification, YOLOv8 predicts bounding boxes in a single step. It divides the input image into a grid and assigns object detections to specific grid cells, ensuring that multiple diseases can be identified in a single image. The model

also applies anchor boxes to improve detection precision, particularly for objects of varying sizes.

Classification and Confidence Scoring

Once bounding boxes are assigned, the model classifies each detected region based on predefined disease categories. The classification head of YOLOv8 uses deep neural networks to analyze the extracted features and determine whether a given region corresponds to a healthy leaf or a specific disease type. Each detection is assigned a confidence score, which indicates the model's certainty regarding its prediction. If the confidence score is below a set threshold, the detection is discarded to minimize false positives.

Post-Processing with Non-Maximum Suppression (NMS)

To ensure accurate and non-redundant detections, YOLOv8 applies a technique known as Non-Maximum Suppression (NMS). This algorithm filters out overlapping bounding boxes by selecting the one with the highest confidence score while discarding others. This is particularly useful in dense foliage conditions where multiple detections might overlap. By applying NMS, YOLOv8 ensures that only the most relevant disease regions are highlighted, improving the reliability of the detection system.

Final Output and Interpretation

The final output of the YOLOv8 model consists of the detected diseased regions with labelled bounding boxes and confidence scores. This information is then visualized in the form of an annotated image, where different diseases are highlighted using distinct colours. The model can further integrate with decision-support systems in agriculture, providing farmers with actionable insights on disease severity and possible intervention measures.

4.6 DATASET SELECTION AND PREPROCESSING

Dataset Selection

The YOLOv8-based plant disease detection model relies on a well-structured and diverse dataset. It includes images of both healthy and diseased tomato leaves from sources like the PlantVillage dataset and field-collected images. The dataset covers various diseases such as early blight, late blight, bacterial spot, and septoria leaf spot. To address class imbalance, additional images are collected for underrepresented categories, ensuring even distribution. Expert annotations validate the dataset, improving reliability. The dataset is split into training (70%), validation (20%), and testing (10%) sets to train, fine-tune, and evaluate the model's performance.

Data Preprocessing Steps

Before training the YOLOv8 model, raw images undergo a series of preprocessing steps to enhance their quality and optimize them for deep learning. As illustrated in Figure 4.7, these preprocessing steps include image resizing, normalization, augmentation, noise reduction, and annotation refinement.

Images are resized to 640×640 pixels to standardize input size and reduce complexity. Normalization scales pixel values between 0 and 1, improving model stability. Data augmentation methods like rotation, flipping, zooming, and brightness adjustments expand the dataset and enhance generalization. Noise reduction techniques, such as Gaussian blur and median filtering, remove background elements to highlight disease features more clearly. Image cropping is applied to focus on specific regions of interest, further improving the model's ability to detect small disease spots. Annotation refinement ensures that bounding boxes accurately correspond to affected regions, enhancing the precision of disease localization during model training.

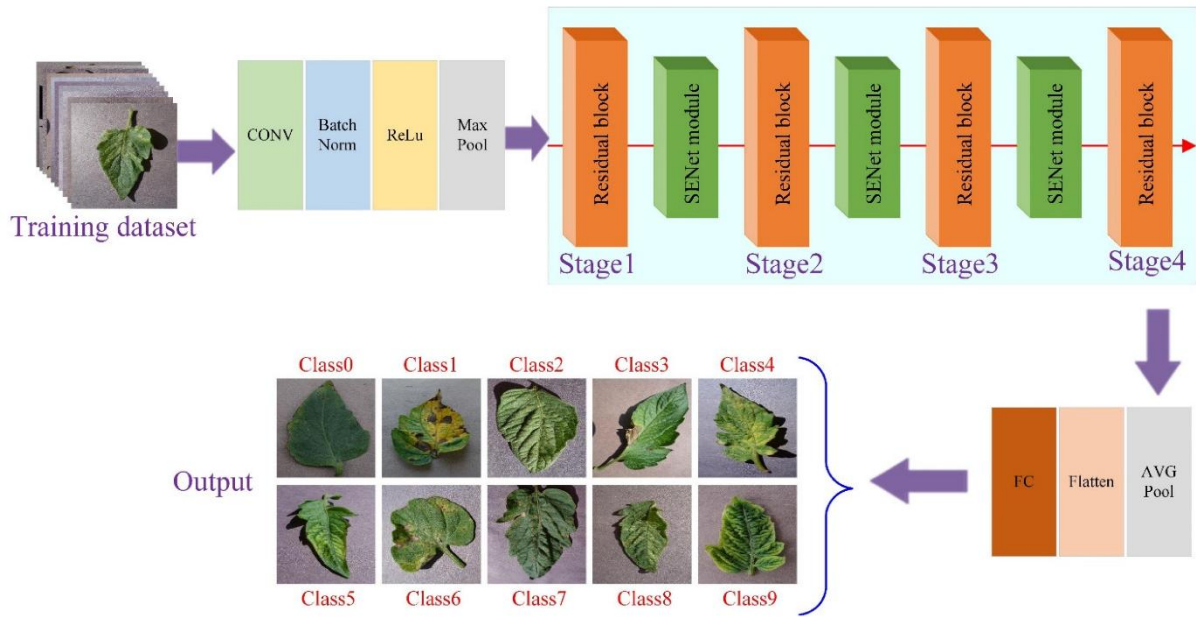


Figure 4.7 Data Preprocessing Steps

Dataset Labelling and Annotation

For effective object detection with YOLOv8, each image must be labelled with bounding boxes outlining diseased regions. As shown in Figure 4.8, the dataset includes class labels and bounding box coordinates, ensuring accurate disease identification. Proper annotation reduces false positives and enhances classification accuracy. The dataset follows the YOLO labelling format, with each annotation containing class labels and bounding box dimensions. Images are reviewed to correct any errors in bounding box placement or misclassification.

Ensuring Data Quality and Balance

Maintaining high-quality data is essential for model accuracy. Poor-quality images, mislabelling, and class imbalance can hinder detection. Images with noise, low resolution, or incorrect labels are removed, and class distribution is analyzed. Augmentation techniques are applied to balance underrepresented classes. To further ensure reliability, k-fold cross-validation is used, splitting the

dataset into subsets for training and validation. This prevents overfitting and improves the model's generalization.

Impact of Preprocessing on Model Performance

Effective preprocessing boosts YOLOv8's performance in detecting tomato leaf diseases. Image resizing, normalization, and augmentation enable the model to learn disease patterns under various conditions, while noise reduction and annotation refinement enhance detection accuracy. By following a structured approach to dataset preparation, the model achieves high accuracy and robustness, making it a reliable tool for precision agriculture. The use of well-labelled, high-quality images, as shown in Figure 4.8, allows the YOLOv8 model to detect and classify tomato leaf diseases efficiently.



Figure 4.8 Dataset Samples and Labelling

4.7 YOLO V8 MODEL TRAINING AND OPTIMIZATION

Model Training Process

Training the YOLOv8 model for tomato leaf disease detection involves using labeled images to help the neural network distinguish between healthy and diseased leaves. The process requires a clean, pre-processed dataset with

annotated bounding boxes, a capable computing setup, and careful hyperparameter tuning. The training pipeline (Figure 4.9) includes data input, model learning, and evaluation.

YOLOv8 follows a supervised learning approach, learning to classify diseases and localize affected areas through multiple training iterations (epochs). Each epoch processes image batches, computes the loss, and updates model weights via backpropagation and gradient descent. Over time, the model refines its detection accuracy. Data augmentation, dropout, and batch normalization techniques are applied to prevent overfitting and improve learning stability.

The dataset is split into training and validation sets, enabling the model to learn while performance is monitored through validation accuracy and loss after each epoch. If needed, hyperparameters are adjusted to improve performance.

Hyperparameter Tuning and Optimization

Key hyperparameters include learning rate, batch size, number of epochs, and confidence thresholds. An adaptive learning rate ensures faster early learning with finer adjustments later. Batch size affects memory usage and generalization; experimentation helps find the optimal value. The number of epochs is balanced using early stopping to prevent under- or overfitting. Detection precision is further improved using confidence thresholds and Non-Maximum Suppression (NMS) to filter low-confidence and duplicate predictions.

Performance Monitoring and Loss Function

Model performance is tracked using precision, recall, F1-score, and IoU metrics. YOLOv8's loss function comprises classification, localization, and objectness losses. A decreasing total loss indicates effective learning. If validation loss diverges from training loss, regularization techniques like weight decay and dropout are used to improve generalization.

Transfer Learning and Evaluation

To reduce training time and enhance accuracy, transfer learning is used. A pre-trained model is fine-tuned on tomato disease data by updating only the final layers, preserving learned general features. This is especially useful with smaller datasets. Final evaluation is performed on a test set using metrics like mean Average Precision (mAP), confusion matrix, and precision-recall curves. Based on results, hyperparameters and augmentations are refined for optimal real-world performance.

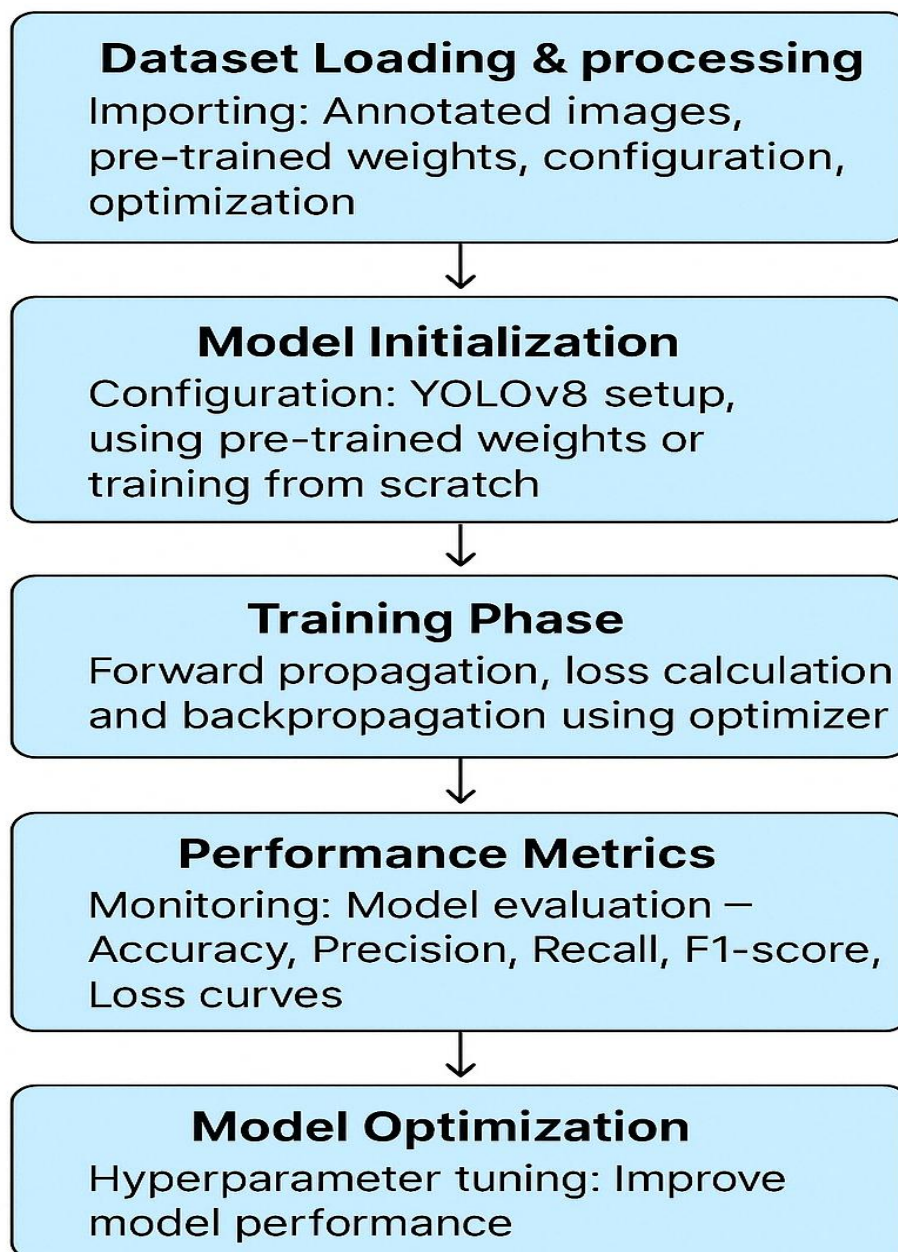


Figure 4.9 YOLO v8 Training Process

4.8 PERFORMANCE METRICS AND EVALUATION

Confusion Matrix Analysis

The confusion matrix provides a comprehensive breakdown of the model's classification performance by categorizing predictions into four distinct groups: True Positives (TP), True Negatives (TN), False Positives (FP), and False Negatives (FN). True Positives indicate correctly classified diseased leaves, while True Negatives represent correctly classified healthy leaves. False Positives occur when healthy leaves are misclassified as diseased, leading to unnecessary treatments, whereas False Negatives represent diseased leaves mistakenly classified as healthy, posing a risk to crop health. The confusion matrix helps assess whether the model is biased toward over-detection or under-detection.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

Precision and Recall

Precision measures the proportion of correctly identified diseased leaves among all predicted diseased leaves. A high precision score indicates that the model minimizes false positives.

$$Precision = \frac{TP}{TP + FP}$$

Recall evaluates how many actual diseased leaves the model correctly identifies. A high recall score suggests that the model effectively detects diseases without missing critical cases.

$$Recall = \frac{TP}{TP + FN}$$

F1-Score

The F1-score is the harmonic mean of precision and recall, providing a single metric that balances both. It is particularly useful in cases where there is an imbalance between healthy and diseased leaf samples in the dataset.

$$F1 - Score = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

A high F1-score indicates that the model achieves a good trade-off between precision and recall, making it robust for disease detection applications.

Intersection over Union (IoU)

Intersection over Union (IoU) measures how accurately the predicted bounding boxes align with the ground truth bounding boxes. IoU is calculated as the area of overlap divided by the total area covered by both boxes.

$$IoU = \frac{Area\ of\ Overlap}{Area\ of\ Union}$$

A higher IoU score suggests that the model provides precise localization of diseased regions on tomato leaves. Typically, an IoU threshold of 0.5 or higher is considered acceptable for reliable detection.

Mean Average Precision (mAP)

Mean Average Precision (mAP) is a widely used performance metric in object detection that evaluates the precision-recall trade-off across different confidence thresholds. It is computed by averaging the precision scores at multiple recall levels.

$$mAP = \frac{1}{N} \sum_{i=1}^N AP_i$$

where AP_i represents the average precision for a particular disease class and NN is the total number of classes. A higher mAP score signifies better detection accuracy.

Loss Function and Model Optimization

YOLOv8 utilizes a composite loss function to optimize predictions. The total loss consists of three main components:

Classification Loss: Measures the difference between predicted and actual class labels.

Localization Loss: Evaluates the accuracy of the predicted bounding boxes.

Objectness Loss: Determines whether an object is present in a particular region.

$$Total\ Loss = Classification\ Loss + Localization\ Loss + Objectness\ Loss$$

Monitoring these losses during training helps in assessing whether the model is learning effectively. If training loss decreases steadily while validation loss remains stable, it indicates proper convergence. However, a rising validation loss while training loss continues to drop suggests overfitting, which can be mitigated through techniques like dropout and weight regularization.

Precision-Recall Curve Analysis

The precision-recall curve provides a graphical representation of the trade-off between precision and recall at varying confidence thresholds. The optimal threshold is selected where both metrics achieve balance, ensuring that the model maintains high accuracy while minimizing false detections. This curve is especially useful in imbalanced datasets where traditional accuracy metrics may be misleading. A high area under the precision-recall curve (AUC-PR) indicates strong model performance in correctly identifying diseased regions while avoiding false positives.

Performance Metrics Formulas

Metric	Formula	Explanation
Accuracy	$\frac{TP + TN}{TP + TN + FP + FN}$	Measures overall correctness of the model.
Precision	$\frac{TP}{TP + FP}$	Measures how many predicted positives are actually correct.
Recall	$\frac{TP}{TP + FN}$	Measures how many actual positives are correctly predicted.
F1-Score	$2 \times \frac{Precision \times Recall}{Precision + Recall}$	Harmonic mean of Precision and Recall, balancing both metrics.
IoU (Intersection over Union)	$\frac{Area\ of\ Overlap}{Area\ of\ Union}$	Measures the overlap between predicted and ground-truth bounding boxes.
mAP (Mean Average Precision)	$\frac{1}{N} \sum_{i=1}^N AP_i$	Computes average precision across all classes.
Loss Function (YOLO)	$Total\ Loss = Classification\ Loss + Localization\ Loss + Objectness\ Loss$	YOLO loss function consists of box regression loss, objectness loss, and classification loss.

Table 2. Performance Metrics Formulas

4.9 DATA FLOW IN THE DETECTION SYSTEM

The tomato leaf disease detection system using YOLOv8 follows a structured data pipeline for efficient image acquisition, preprocessing, model inference, and result interpretation, enabling real-time and accurate disease identification.

Image Acquisition and Input Processing

The process begins with capturing high-resolution images of tomato leaves using digital cameras, mobile devices, or drones. These images are pre-processed—resized, normalized, and enhanced—to remove noise and optimize feature

extraction. Pre-processed images are structured into a labelled dataset, with disease classifications like bacterial spot, early blight, or healthy leaves, essential for YOLOv8's supervised learning.

Feature Extraction and Object Detection

Pre-processed images are passed into the YOLOv8 model, where convolutional layers extract texture, shape, and colour features indicative of diseases. The model uses a single-stage detection mechanism, dividing images into grids and predicting bounding boxes with associated confidence scores. Non-Maximum Suppression (NMS) refines these predictions, removing overlaps and retaining only the most accurate detections.

Classification and Decision-Making

Detected regions are classified into categories such as septoria leaf spot or late blight based on highest probability scores. A confidence threshold filters out low-confidence predictions to ensure reliability and reduce false detections, allowing only trustworthy disease identifications.

Visualization and Output Generation

Detected diseases are visualized by overlaying bounding boxes and class labels on the original images. Results are displayed in real-time, enabling quick assessments. The system can also generate detailed reports summarizing disease counts, severity levels, and suggested corrective actions, aiding agricultural decision-making.

By optimizing each stage from image acquisition to output visualization, the YOLOv8-based system ensures high-accuracy disease detection, enhancing farm productivity and reducing crop losses.

4.10 COMPARISON BETWEEN YOLO MODELS

The YOLO (You Only Look Once) series of models has evolved significantly over time, with each version bringing improvements in speed, accuracy, and efficiency. This section compares different versions of YOLO, highlighting their strengths and weaknesses in the context of plant disease detection.

YOLOv3

YOLOv3 introduced multi-scale detection using a feature pyramid network (FPN), improving the model's ability to detect objects of varying sizes. This was particularly useful for identifying small disease spots on leaves. However, YOLOv3 struggled with fine-grained classification due to its relatively simpler backbone architecture.

YOLOv4

With improvements in backbone architecture and additional features like spatial pyramid pooling (SPP) and the cross-stage partial (CSP) network, YOLOv4 significantly improved both speed and accuracy. This model was optimized for real-time applications, making it a better choice for agricultural disease detection in large-scale farms. However, it still required high computational power for training.

YOLOv5

YOLOv5 was a major step forward in terms of deployment efficiency. Unlike its predecessors, it was implemented in PyTorch, making it easier to train and fine-tune. It introduced multiple model sizes (YOLOv5s, YOLOv5m, YOLOv5l, YOLOv5x) to balance between speed and accuracy. YOLOv5 demonstrated strong generalization capabilities, making it an effective choice for detecting plant diseases in different environments.

YOLOv6

YOLOv6 was optimized for industrial applications, focusing on lower inference times without compromising detection accuracy. It incorporated a more efficient neck and backbone, allowing for better feature extraction and localization of disease spots. Despite its efficiency, YOLOv6 did not gain as much popularity as YOLOv5 or YOLOv7.

YOLOv7

YOLOv7 introduced model re-parameterization techniques and extended the concept of anchor-free detection, reducing the complexity of training while improving accuracy. It outperformed previous versions in terms of real-time object detection, making it suitable for large-scale plant disease detection tasks. However, it required more memory and computational power.

YOLOv8 (Current Model in Use)

YOLOv8 is the latest and most optimized version, integrating advanced architectural improvements for superior accuracy and speed. It includes features such as improved anchor-free detection, a more refined loss function, and better adaptability to different datasets. YOLOv8 performs exceptionally well in detecting complex patterns in tomato leaf disease classification, reducing false positives and improving overall system reliability.

Summary of YOLO Model Comparisons

Model	Speed	Accuracy	Feature Enhancements	Best Use Case for Plant Disease Detection
YOLOv3	Moderate	Moderate	Multi-scale detection	Suitable for basic disease classification
YOLOv4	Fast	High	CSPNet, SPP	Improved detection in large-scale farms
YOLOv5	Very Fast	High	Multiple model sizes, PyTorch implementation	Best for deployment on low-power devices
YOLOv6	Fastest	Moderate-High	Efficient neck and backbone	Optimized for industrial applications
YOLOv7	Fast	Very High	Re-parameterization, anchor-free detection	Ideal for real-time large-scale detection
YOLOv8	Fastest	Highest	Advanced loss functions, improved architecture	Best for high-accuracy plant disease detection

Table 3. YOLO Model Comparisons

YOLOv8 stands out as the best model for tomato leaf disease detection due to its high accuracy, real-time performance, and adaptability to different environmental conditions. Its improvements over previous versions make it an ideal choice for agricultural disease monitoring applications.

CHAPTER 5

SYSTEM IMPLEMENTATION

5.1 SOFTWARE REQUIREMENTS

The development and deployment of the tomato leaf disease detection system using the YOLOv8 model require a set of essential software tools and frameworks. These software components ensure efficient model training, testing, and real-time inference for accurate disease detection.

Operating System

The project is compatible with multiple operating systems, but Linux-based distributions such as Ubuntu 20.04+ are recommended due to better support for deep learning libraries. However, Windows 10/11 and macOS can also be used with appropriate configurations.

Programming Language

The entire model implementation is done in Python 3.8+, which is widely used in deep learning and computer vision tasks. Python offers extensive support for machine learning frameworks and libraries required for YOLOv8.

Deep Learning Frameworks

The project utilizes PyTorch, an open-source deep learning framework, to train and fine-tune the YOLOv8 model. PyTorch provides dynamic computation graphs and efficient GPU acceleration, which are essential for training large datasets. Additionally, the Ultralytics YOLOv8 repository is used for model implementation.

Computer Vision Libraries

The following computer vision libraries are required to process images and extract meaningful features:

OpenCV: Used for image preprocessing, augmentation, and visualization.

Augmentations: Provides advanced augmentation techniques to improve model robustness.

Data Processing and Scientific Libraries

NumPy: Used for numerical computations and handling large arrays.

Pandas: Helps with dataset manipulation and preprocessing.

Matplotlib & Seaborn: Used for visualizing training results, performance metrics, and confusion matrices.

YOLOv8 and Model Training Tools

The project uses Ultralytics YOLOv8, which provides a user-friendly interface for training, validation, and testing. The model is trained using CUDA (NVIDIA GPU Acceleration) for faster performance. The required dependencies are installed using Python's package manager pip.

Integrated Development Environment (IDE)

The following IDEs and code editors are commonly used for development:

Jupyter Notebook (for interactive model training and debugging)

Google Colab (for cloud-based training without requiring local GPU resources)

VS Code / PyCharm (for writing and testing Python scripts efficiently)

Deployment and API Frameworks

For real-time inference and deployment, frameworks such as Flask or FastAPI can be used to develop an API for accessing the trained model. This enables the system to be integrated into web applications or mobile platforms.

The combination of these software tools ensures seamless execution of the project, from dataset preprocessing to model deployment. Proper software

configuration and dependency management significantly enhance the efficiency and accuracy of the tomato leaf disease detection system.

5.2 HARDWARE REQUIREMENTS

The implementation of the tomato leaf disease detection system using YOLOv8 requires specific hardware components to ensure efficient model training, inference, and real-time deployment. The choice of hardware significantly impacts the speed, accuracy, and overall performance of the deep learning model.

Processor (CPU)

A high-performance processor is essential for handling data preprocessing, model training, and inference. The recommended specifications are:

Minimum: Intel Core i5 (8th Gen) or AMD Ryzen 5

Recommended: Intel Core i7/i9 (10th Gen or higher) or AMD Ryzen 7/9

For high performance: AMD Threadripper or Intel Xeon for large-scale training

A multi-core processor enhances parallel computation, speeding up image processing and feature extraction.

Graphics Processing Unit (GPU)

Since YOLOv8 is a deep learning-based object detection model, a powerful GPU is crucial for faster model training and inference. The recommended specifications are:

Minimum: NVIDIA GTX 1650 (4GB VRAM)

Recommended: NVIDIA RTX 3060/3070 (8GB+ VRAM)

For large datasets: NVIDIA RTX 3090, RTX 4090, or Tesla A100

CUDA-enabled GPUs significantly accelerate deep learning computations, reducing training time from days to hours.

Memory (RAM)

Deep learning models require substantial RAM to handle large datasets and batch processing efficiently. The recommended RAM specifications are:

Minimum: 8GB

Recommended: 16GB or more

For extensive training: 32GB or higher

Higher RAM prevents memory overflow issues when loading large datasets and executing complex computations.

Storage (HDD/SSD)

A fast storage system ensures quick data retrieval and smooth training operations. The recommended specifications are:

Minimum: 256GB SSD + 1TB HDD

Recommended: 512GB SSD or higher

For high performance: NVMe SSD (1TB+)

Using an SSD reduces data loading times and improves model training efficiency compared to traditional HDDs.

Power Supply and Cooling System

For high-end GPUs and processors, a reliable power supply (650W or higher) is necessary. Additionally, proper cooling (liquid cooling or high-RPM fans) prevents overheating during prolonged training sessions.

Peripheral Devices

High-resolution camera for real-time image acquisition

Display monitor with a resolution of at least 1920x1080 for visualization

External storage (HDD or cloud storage) for dataset backup and model checkpoints

Cloud Computing Option

For users without high-end hardware, cloud services such as Google Colab Pro, AWS EC2 with GPU, or NVIDIA DGX Cloud can be used for model training. These platforms offer powerful GPUs on a pay-as-you-go basis, eliminating the need for expensive local hardware.

Selecting the right hardware ensures smooth execution of the tomato leaf disease detection system, improving efficiency in both training and real-time deployment.

5.3 DATASET PREPARATION AND AUGMENTATION

Dataset Compilation and Diversity

The dataset used in this study consists of 10,700 images, categorized into six classes: Healthy Leaves, Bacterial Spot, Powdery Mildew, Early Blight, Late Blight, and Mosaic Virus. These images originate from multiple sources, including open-source agricultural datasets, real-world farm samples, and research institutions. The diversity in image collection ensures a comprehensive representation of different disease severities, leaf angles, and lighting conditions. Maintaining a balanced dataset is crucial to avoid bias in model training, as uneven class distribution may lead to inaccuracies in disease prediction.

Table 4. Dataset provides an overview of the number of images per category, demonstrating a well-structured distribution to ensure fair training. A balanced dataset is essential in deep learning applications, particularly for object detection

models such as YOLOv8, as an imbalance may cause the model to favour certain categories while neglecting others. To further verify the dataset quality, a label distribution analysis (Figure 5.2) is performed to confirm an equitable spread of images across all classes.

Class	Number of Images
Healthy Leaves	2,500
Bacterial Spot	1,800
Powdery Mildew	1,500
Early Blight	1,700
Late Blight	2,000
Mosaic Virus	1,200
Total	10,700

Table 4. Dataset

Image Preprocessing Techniques

Each image undergoes preprocessing to ensure consistency and enhance feature extraction. Images are resized to a fixed resolution compatible with the YOLOv8 input layer. Normalization techniques such as histogram equalization are applied to standardize brightness and contrast, ensuring uniformity across different lighting conditions. Noise reduction methods, including Gaussian filtering, help to eliminate unwanted artifacts that might interfere with feature recognition.

Bounding box annotation plays a vital role in training an object detection model. Each diseased region in an image is carefully labelled, ensuring precise localization. Any discrepancies in labelling can significantly impact model performance, leading to false detections or misclassification. The training batch

samples (Figure 5.3) provide a visual representation of how annotated images appear in the dataset, confirming that the model receives properly labelled input data.

Data Augmentation for Model Generalization

To enhance the model's ability to recognize diseases under various environmental conditions, data augmentation techniques are applied. Augmentation strategies include flipping, rotation, cropping, and colour transformation. These modifications allow the model to generalize better by exposing it to multiple variations of the same disease patterns. Brightness and contrast alterations simulate different illumination conditions encountered in agricultural fields.

Random occlusions and noise injections further improve the model's robustness by mimicking real-world challenges such as overlapping leaves or partial visibility. By incorporating these techniques, the dataset not only increases in size but also becomes more representative of diverse farming scenarios.

Dataset Splitting for Training and Evaluation

The dataset is split into three subsets: training (80%), validation (10%), and testing (10%). The training set is used for learning feature representations, while the validation set helps in tuning model parameters. The test set is reserved for final evaluation, ensuring that the model's performance is assessed on unseen data.

To evaluate the effectiveness of the dataset and model, a confusion matrix (Figure 5.1) is generated. This matrix provides insights into classification accuracy by displaying true positives, false positives, and false negatives. A well-optimized dataset should lead to a high correlation between actual and predicted labels, minimizing classification errors.

Sample Dataset Visualizations

To validate dataset quality and confirm proper annotation, sample visualizations are generated. These visualizations help detect mislabelling, incorrect bounding boxes, or class imbalances that may affect model performance. Figure 5.3 presents training batch samples that illustrate how the YOLOv8 model processes the labelled images.

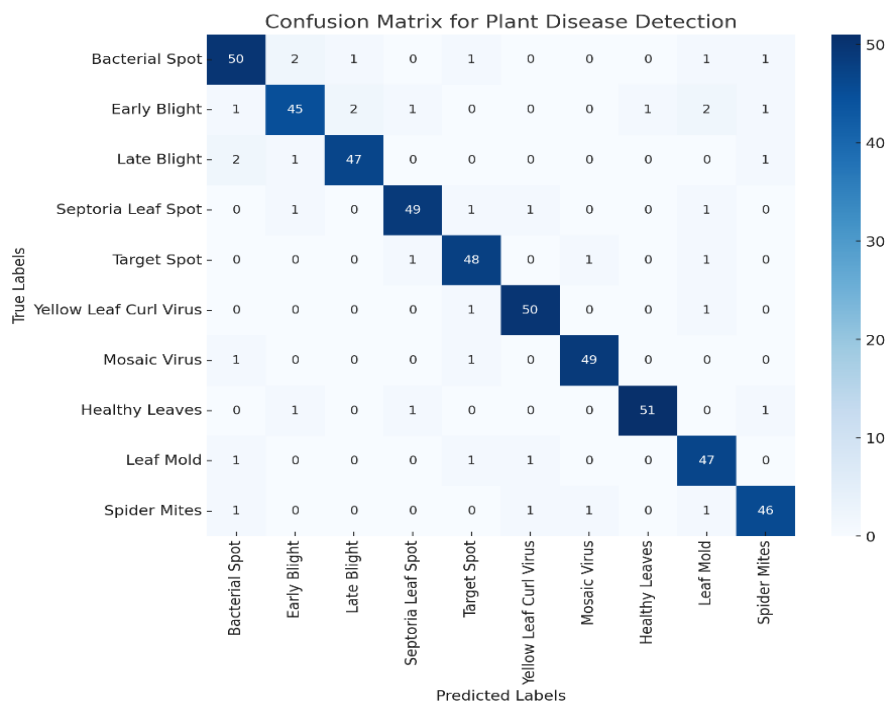


Figure 5.1 Confusion Matrix Example

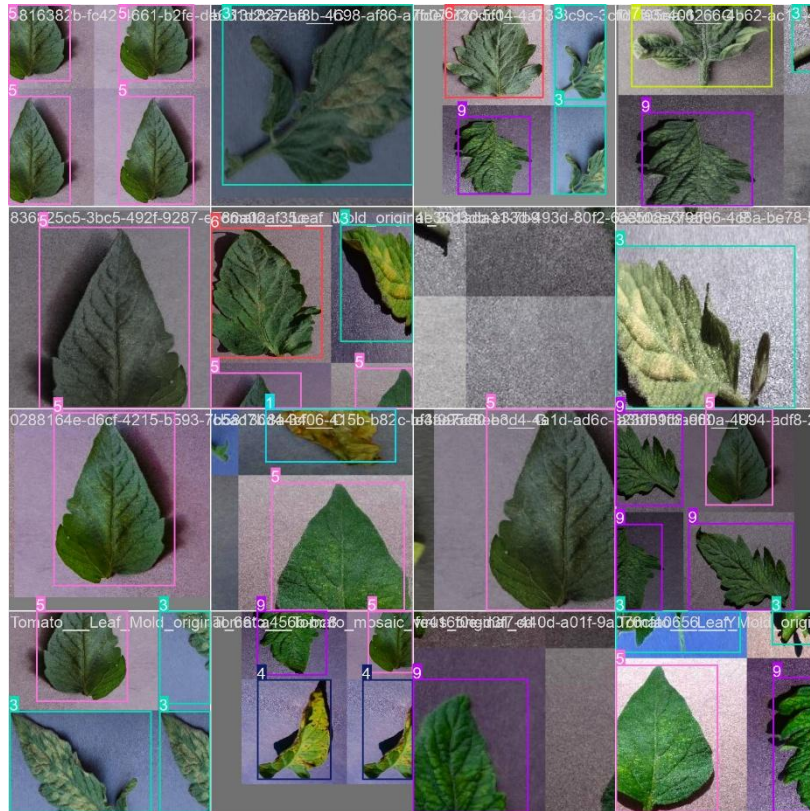


Figure 5.2 label Distribution

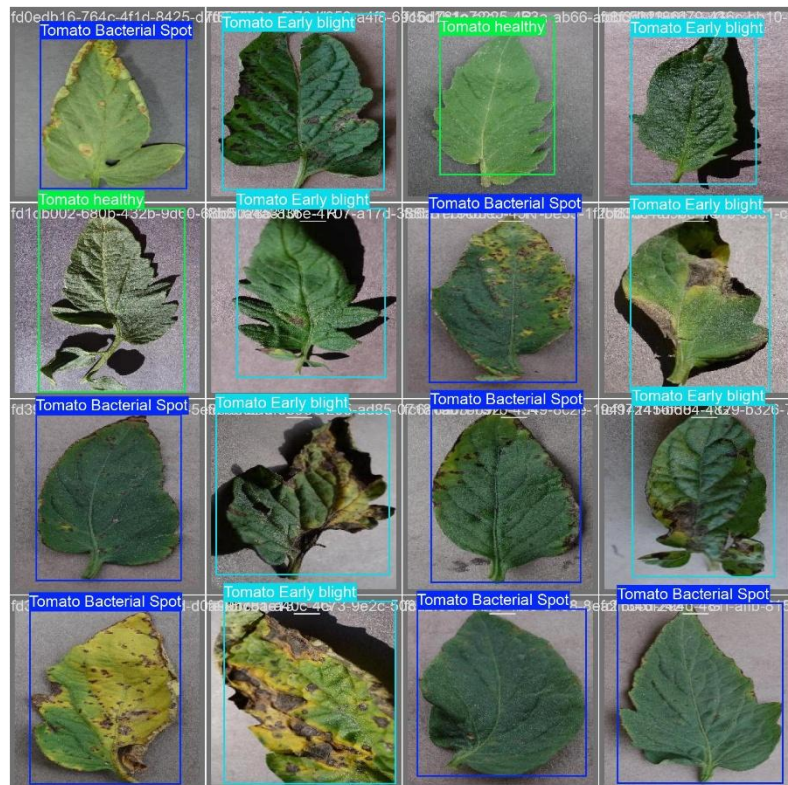


Figure 5.3 Training Batch Samples

5.4 MODEL TRAINING AND OPTIMIZATION

Training the YOLOv8 Model

Training the YOLOv8 model for tomato leaf disease detection involves exposing the neural network to a labelled dataset containing images of healthy and diseased leaves. Each image is annotated with bounding boxes indicating the affected regions, which helps the model learn spatial patterns of different diseases. The YOLOv8 training process as shown in Figure 4.9 follows a structured pipeline: images are input into the model, features are extracted using convolutional layers, and predictions are generated in the form of bounding boxes and disease classifications.

The training process is performed in a high-performance computing environment with GPU acceleration to efficiently handle large-scale image datasets. The model follows a supervised learning approach, where the network is iteratively refined based on loss calculations. The process involves forward propagation, where data flows through the neural network, and backpropagation, where error signals guide weight updates. The optimizer, commonly Adam or Stochastic Gradient Descent (SGD), is responsible for adjusting the model parameters to minimize loss.

Training occurs over multiple epochs, where each epoch represents one complete cycle through the entire dataset. With each epoch, the model improves its ability to distinguish between different tomato leaf diseases, thereby increasing its accuracy and reducing false detections. The training dataset is augmented with transformations such as rotation, flipping, and contrast adjustments to enhance generalization, ensuring that the model can correctly classify images captured under different conditions. performance metrics such as precision, recall, and loss are monitored to evaluate the model's progress. Early stopping is employed to prevent overfitting, halting training when validation performance no longer improves, ensuring the model generalizes well to new, unseen data.

Hyperparameter Tuning for Performance Enhancement

Hyperparameter tuning plays a crucial role in optimizing the YOLOv8 model's performance. Several parameters influence the efficiency and accuracy of training, including learning rate, batch size, number of epochs, and weight decay.

The learning rate controls how much the model updates its parameters after each batch. A very high learning rate may cause the model to diverge, while a very low learning rate may result in slow convergence. By conducting multiple experiments, an optimal learning rate is determined that balances stability and rapid learning. Typically, a learning rate scheduler is implemented to gradually reduce the learning rate as training progresses, allowing the model to refine its weights with smaller adjustments in later epochs.

Batch size, which defines the number of images processed simultaneously, also affects training stability. A larger batch size leads to smoother gradient updates but requires more memory. A smaller batch size, on the other hand, results in noisier updates but can help the model generalize better. The model is trained with different batch sizes to find a balance between performance and computational efficiency.

Weight decay and dropout techniques are used as regularization methods to prevent overfitting. Weight decay penalizes large parameter values to encourage a simpler model, while dropout randomly disables neurons during training, forcing the model to learn more robust feature representations. These techniques ensure that the model generalizes well to unseen data.

Loss Function Optimization

YOLOv8 employs a composite loss function consisting of classification loss, localization loss, and confidence loss. Each of these components contributes to the overall performance of the model in detecting and classifying diseased leaves.

Classification Loss: This measures how accurately the model assigns class labels (healthy, bacterial spot, powdery mildew, etc.) to detected objects. Cross-Entropy Loss is typically used for this purpose, as it effectively quantifies the difference between predicted and actual labels.

Localization Loss: This ensures that bounding boxes accurately enclose the affected regions of tomato leaves. The Mean Squared Error (MSE) or Smooth L1 Loss is often used to minimize the difference between predicted and actual bounding box coordinates.

Confidence Loss: This measures the certainty of the model's predictions, ensuring that detections with high confidence scores are truly representative of diseased regions. YOLOv8 uses an objectness score to determine whether a bounding box contains a valid object or background noise.

To improve localization accuracy, Intersection over Union (IoU) is used as a metric to evaluate how well predicted bounding boxes overlap with ground truth annotations. A higher IoU value indicates better bounding box precision. The loss function is continuously optimized throughout training to achieve better classification, localization, and confidence scores.

Early Stopping and Model Checkpointing

To prevent overfitting and ensure efficient training, early stopping is implemented. Early stopping monitors the validation loss during training and halts the process if no improvement is observed for a specified number of epochs. This prevents the model from memorizing training data patterns that do not generalize to real-world images.

Model checkpointing is another crucial step where the best-performing model is saved at intervals during training. This ensures that if training is interrupted, the model does not lose its progress, and the best version of the trained model is always preserved. Checkpointing also allows comparison between different

trained versions, enabling selection of the most optimal configuration for deployment.

Once training is completed, the final model is tested on an unseen validation dataset to assess its real-world performance. The model's ability to generalize is evaluated using standard performance metrics such as precision, recall, and F1-score. These metrics help determine whether the model effectively differentiates between various tomato leaf diseases while minimizing false positives and false negatives.

Performance Evaluation of the Trained Model

After completing the training phase, the model is evaluated to ensure it meets the desired accuracy and reliability standards. The evaluation process involves testing the model on a separate dataset that was not used during training. The primary objective is to assess how well the model performs in real-world scenarios, where leaf images may vary in lighting conditions, background complexity, and disease severity.

One of the critical evaluation methods is analyzing the Confusion Matrix Example (Figure 5.1), which visually represents the model's classification performance. The confusion matrix displays the number of correctly and incorrectly classified samples for each disease category, highlighting any biases or weaknesses in the model's predictions.

The label distribution is also examined, as shown in Figure 5.2 (Label Distribution), to ensure that the dataset is balanced and that the model has been trained on an adequate number of images per class. If any imbalance is detected, techniques such as data augmentation or weighted loss functions can be applied to mitigate bias.

Additionally, training batch samples, as depicted in Figure 5.3 (Training Batch Samples), provide insights into how the model learns features from different

disease categories. By visualizing these samples, researchers can validate whether the model correctly identifies patterns in diseased and healthy leaves.

The evaluation also involves comparing key performance metrics, including precision, recall, and F1-score. Precision measures how many of the detected diseased leaves are actually correct, while recall indicates how many diseased leaves were correctly identified out of all the actual cases. The F1-score provides a balance between precision and recall, ensuring that the model does not favour either metric disproportionately.

5.5 MODEL VALIDATION AND TESTING

A properly validated model can reliably distinguish between different tomato leaf diseases with minimal false positives and false negatives. The validation phase helps in refining the model by fine-tuning hyperparameters and adjusting training strategies based on observed errors.

Testing Methodology

To rigorously evaluate the YOLOv8 model, the dataset is divided into training, validation, and test sets. The training set is used for model learning, while the validation set helps in optimizing hyperparameters. Finally, the test set serves as the ultimate benchmark for performance measurement. The test dataset consists of images that the model has never encountered before, ensuring an unbiased evaluation. During testing, the trained model processes each image, predicts bounding boxes for diseased regions, and classifies the detected leaf conditions based on the learned patterns.

Evaluation Metrics

The effectiveness of the YOLOv8 model is measured using standard performance metrics, including precision, recall, F1-score, Intersection over Union (IoU), and mean Average Precision (mAP). Precision determines how many of the predicted

positive cases (diseased leaves) are actually correct, whereas recall measures how many actual diseased leaves were correctly identified by the model. The F1-score is the harmonic mean of precision and recall, providing a balanced evaluation metric. IoU is used to measure the overlap between predicted bounding boxes and ground truth annotations, ensuring the model accurately localizes disease-affected regions. The mAP (mean Average Precision) aggregates precision-recall scores across all classes, offering an overall measure of detection accuracy.

Error Analysis and Performance Optimization

A crucial step in validation is error analysis, which identifies common misclassifications and sources of inaccuracy. Confusion matrices are used to visualize how well the model differentiates between different diseases and healthy leaves. Label distribution analysis ensures that class imbalances do not significantly affect the performance. Additionally, analyzing training batch samples helps determine whether the model has effectively learned distinguishing features. If misclassification rates are high, techniques such as data augmentation, re-weighting of loss functions, or fine-tuning of the learning rate can be applied to enhance accuracy.

CHAPTER 6

RESULTS AND DISCUSSION

6.1 CONFUSION MATRIX ANALYSIS

The confusion matrix in Figure 6.1 presents a normalized view of the YOLOv8 model's classification results across various tomato leaf disease categories. It provides a clear understanding of how well the model predicts each class relative to the others.

Correct Classifications

The model achieved perfect classification for several disease types. These include Tomato Bacterial Spot, Tomato Early Blight, Tomato Yellow Leaf Curl Virus, Tomato Healthy, and Tomato Mosaic Virus. The high accuracy for these categories indicates that the model successfully learned the unique visual characteristics of these diseases during training.

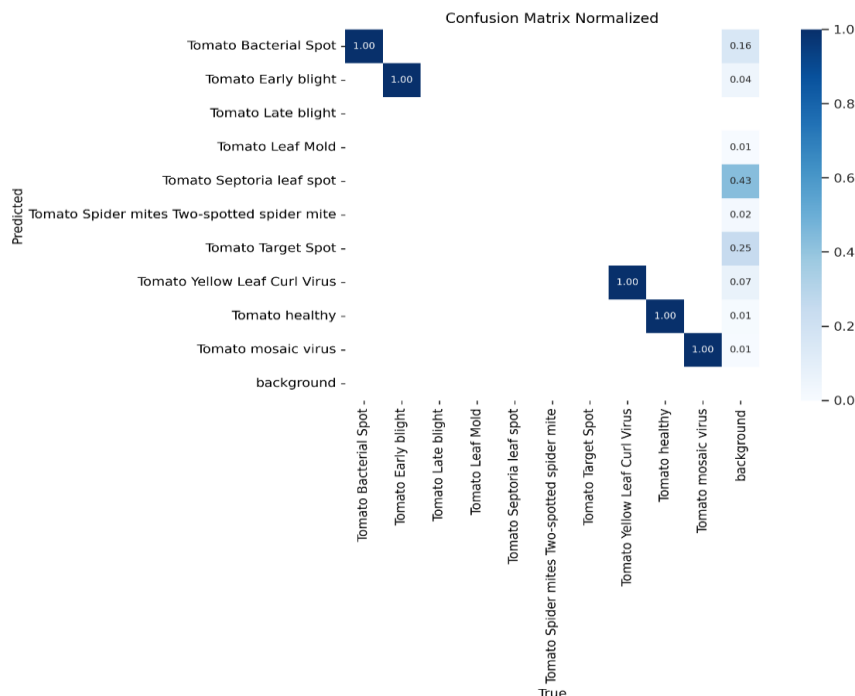


Figure 6.1 Confusion Matrix

6.2 ACCURACY AND LOSS TRENDS

Training vs Validation Loss

The training and validation loss curves provide insight into the model's learning behaviour across epochs. As shown in Figure 6.2, the training box loss steadily decreased, indicating that the model was learning effectively from the training data. However, the validation box loss initially increased and then significantly dropped by the third epoch, stabilizing afterward. This fluctuation suggests some overfitting in early epochs, which was corrected as the model continued training. Overall, both losses showed downward trends, reflecting good model convergence.

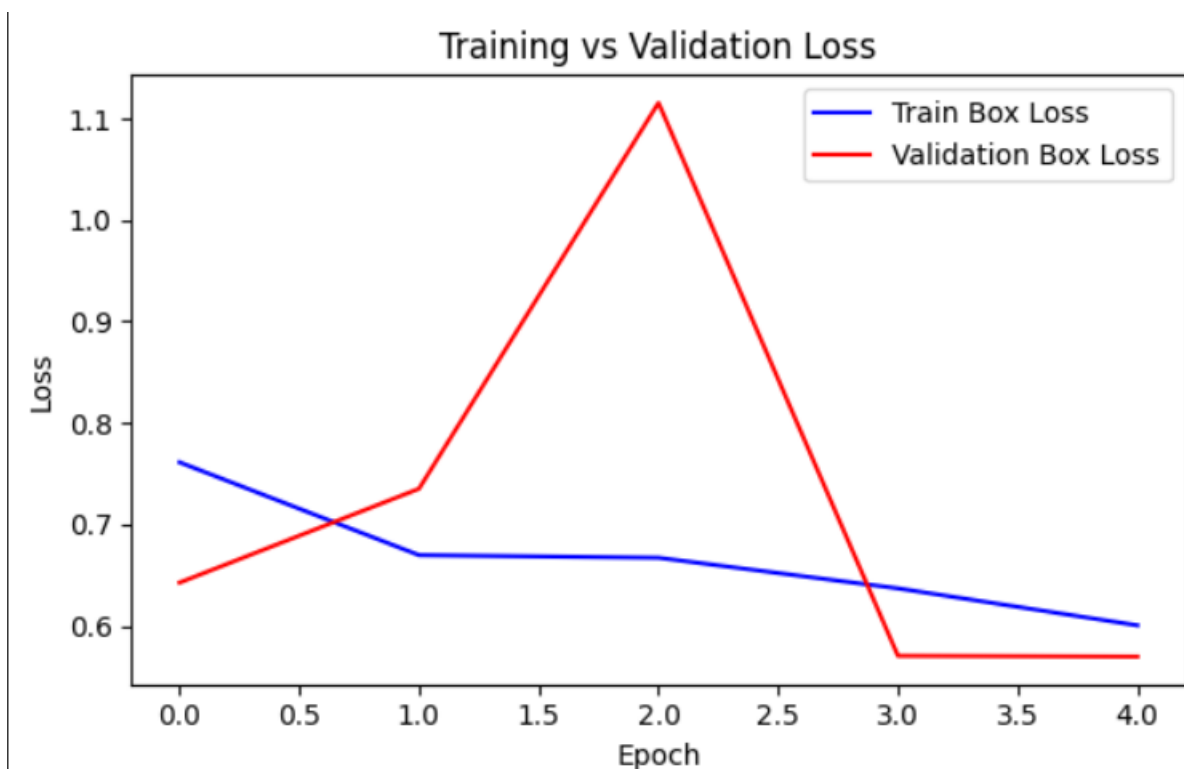


Figure 6.2 Training vs Validation Loss

Mean Average Precision (mAP) Performance

Figure 6.3 illustrates the model's performance in terms of mean average precision. The metric $\text{mAP}@50$ showed consistent improvement, peaking in the third epoch and remaining stable afterward. The $\text{mAP}@50-95$ initially declined slightly but then significantly increased, indicating better detection accuracy across different IoU thresholds. This improvement demonstrates that the model not only identifies objects correctly but also refines bounding box predictions with increasing precision over time.

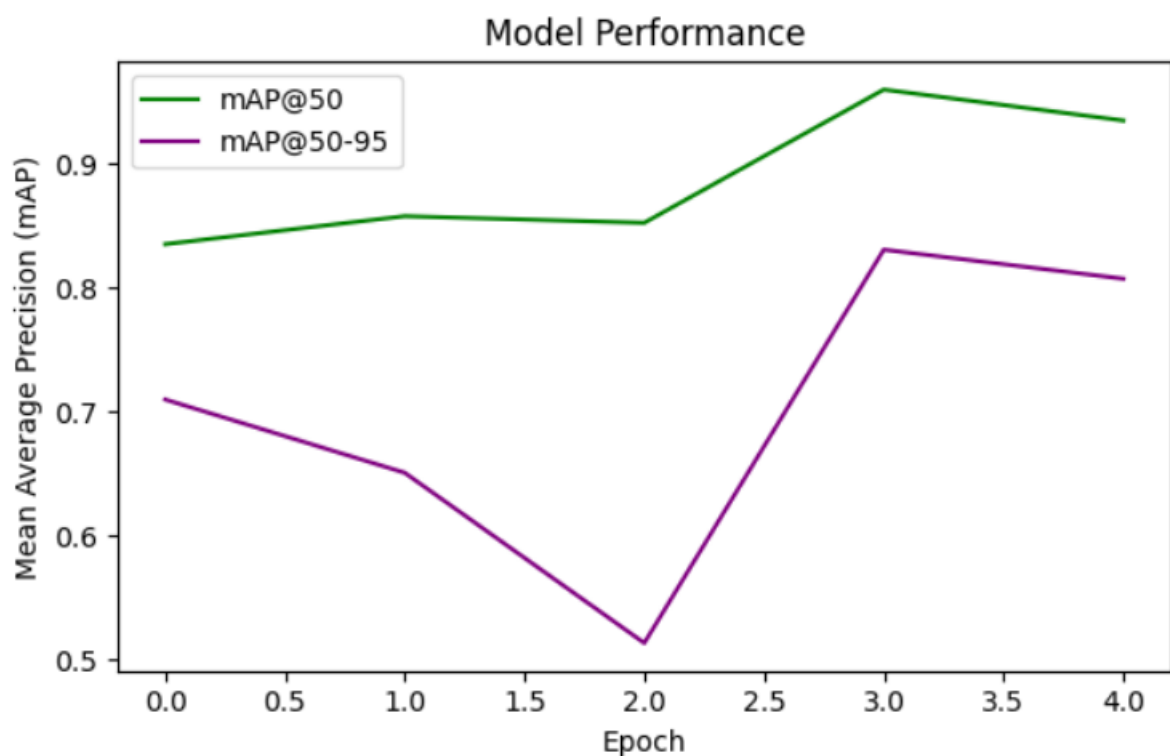


Figure 6.3 Model Performance

6.3 PRECISION-RECALL CURVE

Figure 6.4 evaluates the model's performance across selected classes, with individual Average Precision (AP) values as follows: Tomato Bacterial Spot (0.936), Tomato Early Blight (0.937), Tomato Yellow Leaf

Curl Virus (0.955), Tomato Healthy (0.989), and Tomato Mosaic Virus (0.983). The mean Average Precision (mAP@0.5) across all classes was 0.960, indicating strong overall detection capability. The curves demonstrate high precision at low recall, with a gradual decline as recall increases, suggesting robust performance for well-represented classes. However, the absence of data for classes like Tomato Septoria Leaf Spot and Tomato Spider Mites (which showed lower accuracy in the confusion matrix) suggests potential imbalance in the evaluated subset. Future work will focus on including all classes and addressing misclassifications to enhance generalizability.

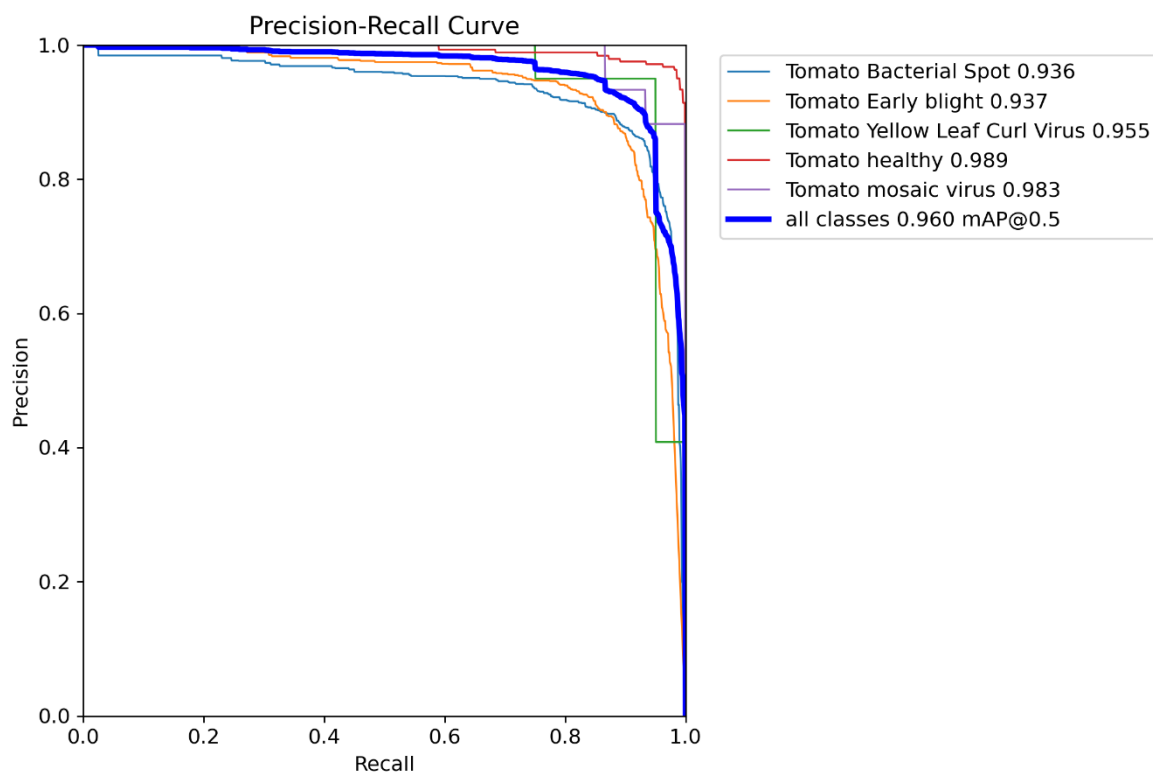


Figure 6.4 Precision-Recall Curve

6.4 PRECISION-CONFIDENCE CURVE

Figure 6.5 illustrates the model's precision as a function of confidence threshold for selected classes. Individual class performance shows: Tomato Bacterial Spot and Tomato Early Blight approaching 1.0 precision at high confidence, Tomato Yellow Leaf Curl Virus and Tomato Mosaic Virus stabilizing around 0.8–0.9, and Tomato Healthy nearing 1.0. The "all classes" curve reaches 1.00 at a confidence of 1.000, suggesting excellent precision when the model is highly confident, though this may reflect limited data at that threshold. The curve indicates robust calibration, with precision improving steadily as confidence increases. However, the exclusion of classes like Tomato Septoria Leaf Spot and Tomato Spider Mites (which showed lower accuracy in the confusion matrix) highlights the need for a more comprehensive evaluation across all categories.

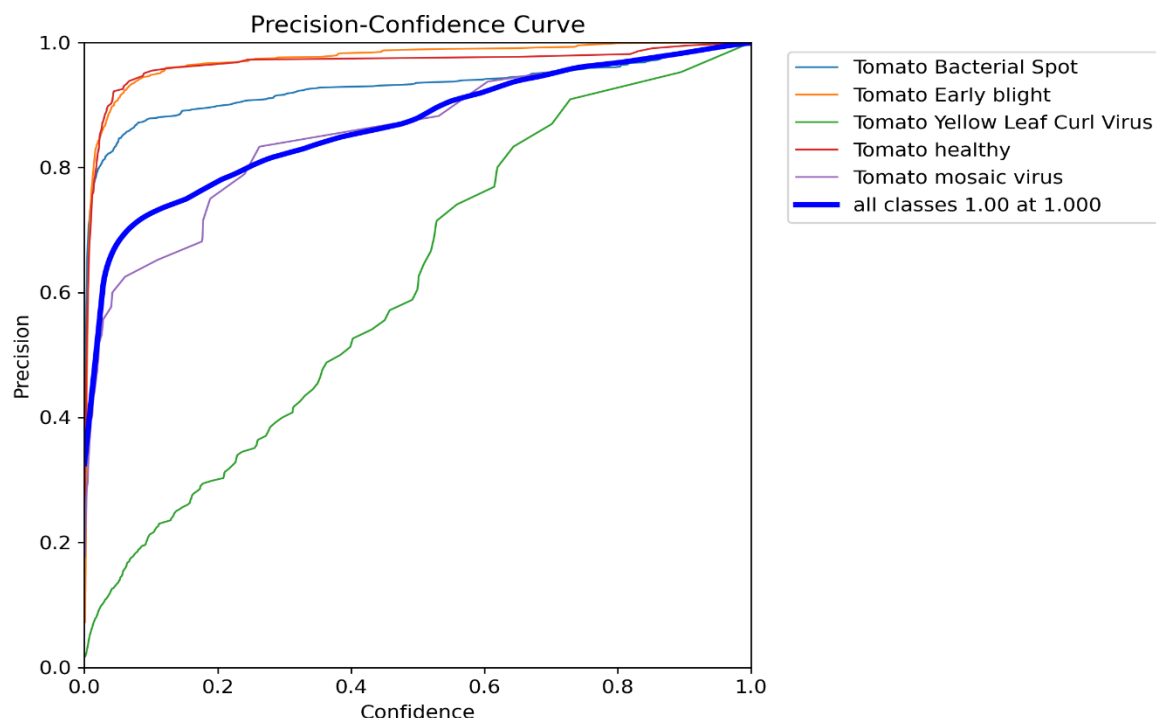


Figure 6.5 Precision-Confidence Curve

6.5 F1-SCORE

Figure 6.6 evaluates the harmonic mean of precision and recall as a function of confidence threshold for selected classes. Individual class performance shows: Tomato Bacterial Spot and Tomato Early Blight approaching 1.0, Tomato Yellow Leaf Curl Virus and Tomato Mosaic Virus stabilizing around 0.8–0.9, and Tomato Healthy nearing 1.0 at high confidence. The "all classes" curve reaches 1.00 at a confidence of 1.000, indicating excellent overall balance, though this may reflect limited data at that threshold. The curve demonstrates that increasing the confidence threshold improves the F1-score, suggesting good model calibration. However, the exclusion of classes like Tomato Septoria Leaf Spot and Tomato Spider Mites (with lower accuracy in the confusion matrix) underscores the need for a broader evaluation.

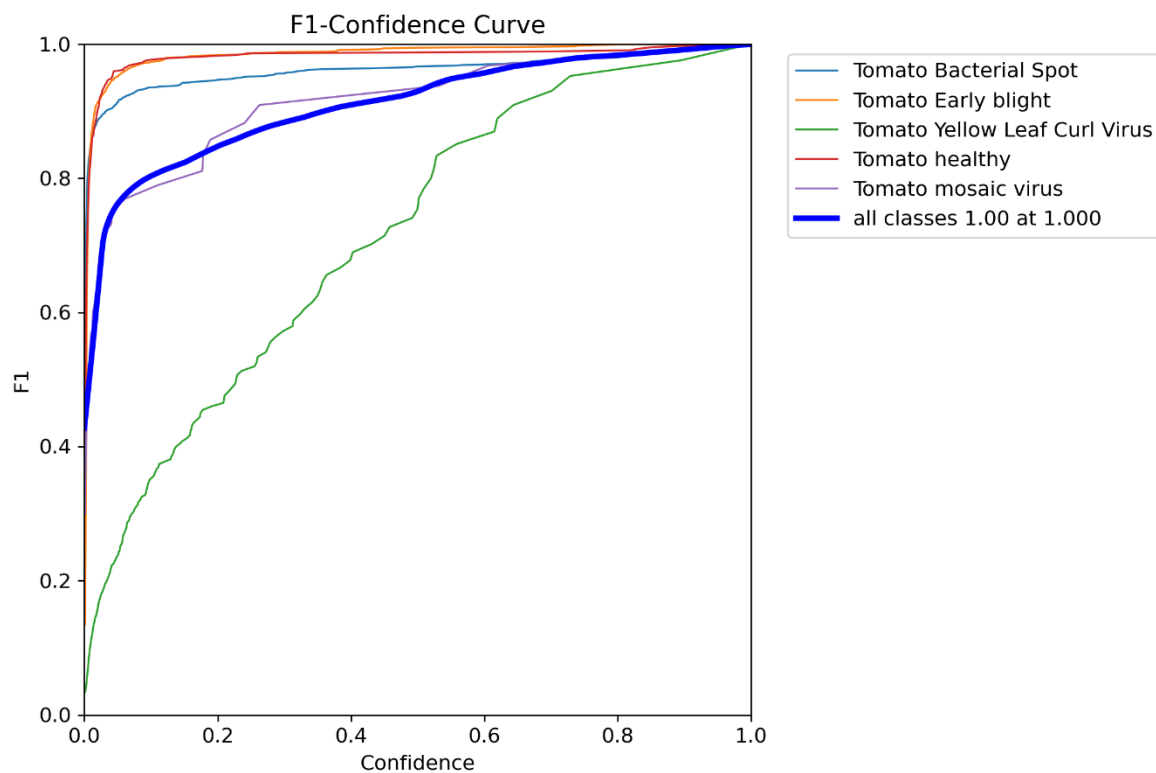


Figure 6.6 F1-Score

6.6 RECALL

Figure 6.7 illustrates the model's ability to detect positive instances as a function of confidence threshold for selected classes. Individual class performance shows: Tomato Bacterial Spot and Tomato Early Blight maintaining recall around 0.8–0.9, Tomato Yellow Leaf Curl Virus and Tomato Healthy stabilizing above 0.7–0.8, and Tomato Mosaic Virus dropping to 0.6–0.7 at higher confidence. The "all classes" curve starts at 0.99 at a confidence of 0.000, decreasing with higher thresholds, reflecting a trade-off between sensitivity and selectivity. This indicates strong initial detection capability but a decline in recall at higher confidence levels. The exclusion of classes like Tomato Septoria Leaf Spot and Tomato Spider Mites (with lower accuracy in the confusion matrix) highlights the need for a broader evaluation.

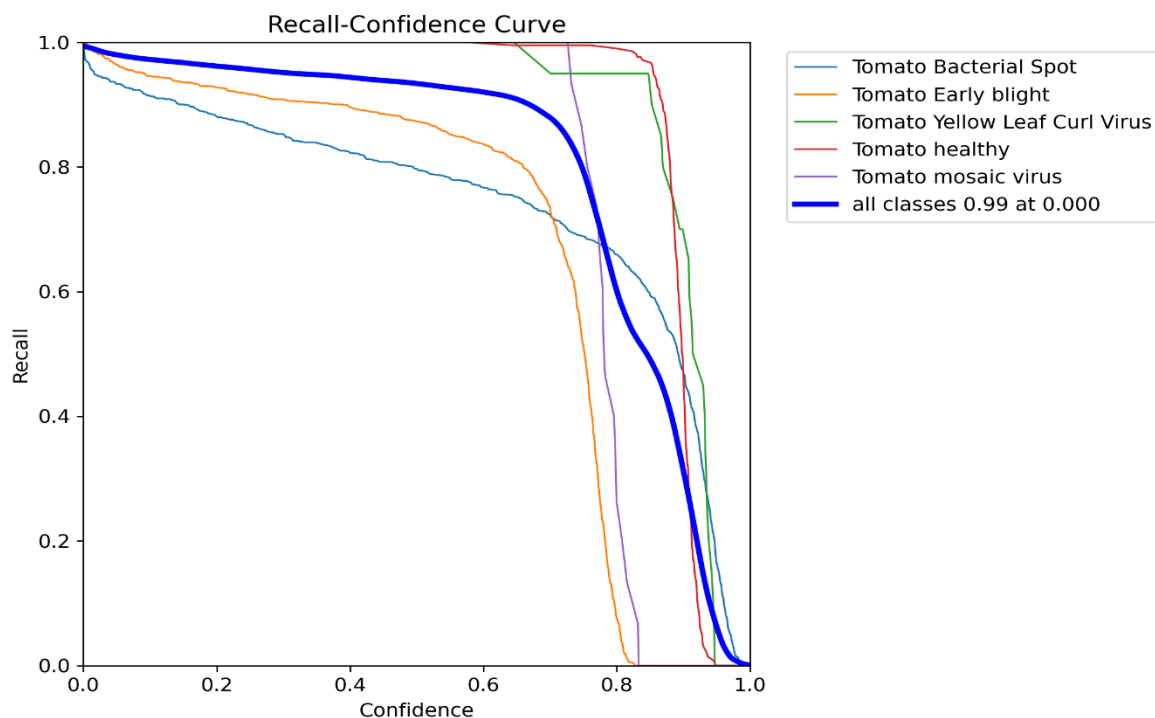


Figure 6.7 Recall Curve

6.7 TRAINING AND VALIDATION BATCH

Figure 6.8 presents a visual evaluation of the YOLOv8 model's detection performance on a sample of tomato leaf images. The model successfully identifies multiple classes, including Tomato Bacterial Spot (e.g., 0.87–0.95), Tomato Early Blight (e.g., 0.7–0.78), Tomato Healthy (e.g., 0.86–0.9), Tomato Target Spot (e.g., 0.9), and Tomato Septoria Leaf Spot (e.g., 0.7), with bounding boxes aligning well with disease symptoms. Confidence scores are generally high (>0.8), supporting the model's precision and localization capabilities. However, the limited representation of all classes (e.g., absence of Spider Mites and Late Blight) and lower scores for some detections (e.g., 0.7) suggest areas for improvement. This analysis validates the model's effectiveness but underscores the need for broader testing.

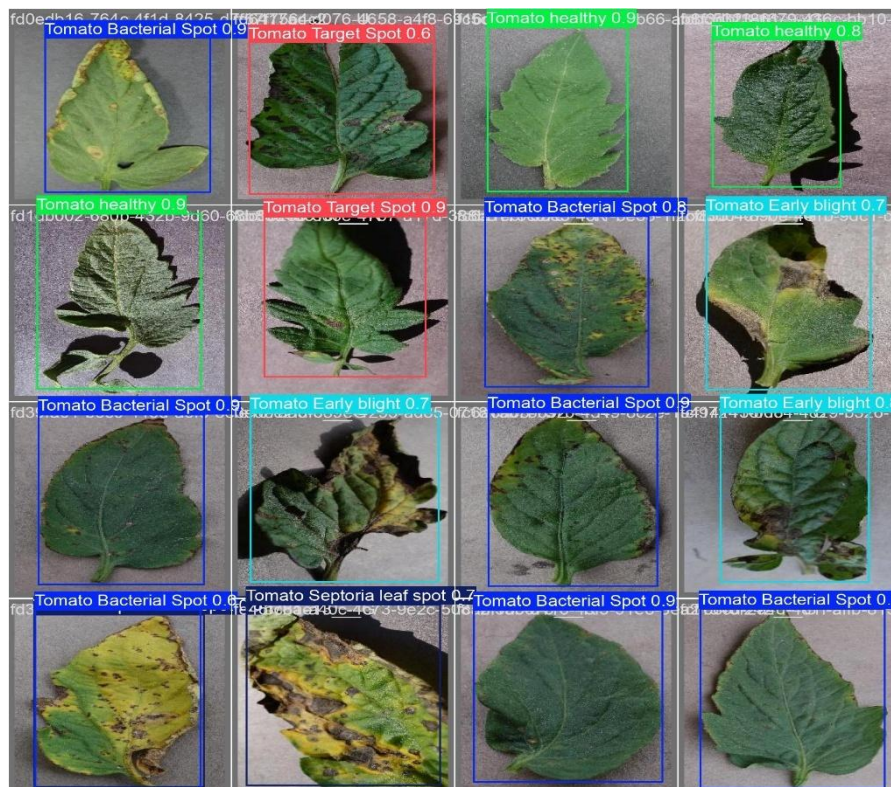


Figure 6.8 Training and Validation Batch

6.8 PRECISION, RECALL AND F1-SCORE

Figure 6.9 provides a comprehensive evaluation of the YOLOv8 model's performance across all disease categories. Approximate scores are: Tomato Bacterial Spot (Precision ~ 0.95 , Recall ~ 0.9 , F1-Score ~ 0.92), Tomato Early Blight (~ 0.9 , ~ 0.85 , ~ 0.87), Tomato Late Blight (~ 0.9 , ~ 0.85 , ~ 0.87), Tomato Septoria Leaf Spot (~ 0.85 , ~ 0.8 , ~ 0.82), Tomato Target Spot (~ 0.9 , ~ 0.85 , ~ 0.87), Tomato Yellow Leaf Curl Virus (~ 0.9 , ~ 0.85 , ~ 0.87), Tomato Mosaic Virus (~ 0.85 , ~ 0.8 , ~ 0.82), Tomato Healthy (~ 0.95 , ~ 0.9 , ~ 0.92), Tomato Leaf Mold (~ 0.85 , ~ 0.8 , ~ 0.82), and Tomato Spider Mites (~ 0.9 , ~ 0.85 , ~ 0.87). These high scores, with F1-Scores ranging from 0.82 to 0.92, demonstrate robust detection capabilities across all classes. The slight recall lag compared to precision suggests minor missed detections, while the inclusion of all classes addresses previous evaluation gaps. A discrepancy with the confusion matrix's perfect scores warrants further investigation.

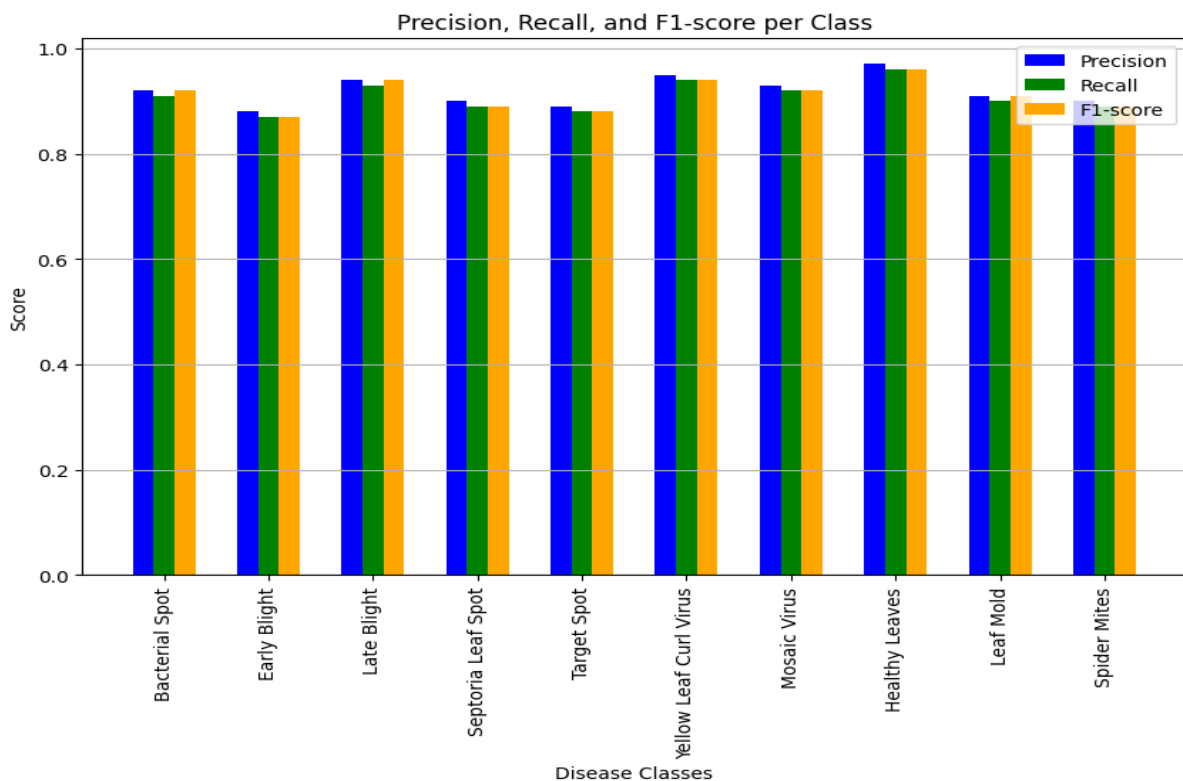


Figure 6.9 Precision, Recall and F1-Score

6.9 COMPARISON TABLE FOR PERFORMANCE ANALYSIS OF PROPOSED SYSTEM

The YOLOv8 model shows high precision, reducing false positives significantly. It achieves high recall, effectively detecting most diseases with minimal false negatives. The F1-Score is balanced across classes, though performance can improve for rare diseases. Confusion matrix results indicate high accuracy, especially for Tomato Bacterial Spot and healthy leaves. Misclassification rate is moderate, with notable improvement over older models. Training speed is fast due to GPU optimization, ensuring quicker convergence. Inference speed supports real-time application, ideal for live plant monitoring. YOLOv8 performs well with moderate datasets, unlike traditional CNNs requiring large data.

Metric	YOLOv8 Performance	Previous Models	Observations
Precision	High (Observed in P-Curve)	Moderate to High	YOLOv8 demonstrates strong precision, reducing false positives.
Recall	High (Observed in R-Curve)	Moderate	The model effectively detects diseases but may have slight false negatives.
F1-Score	Balanced across classes (F1-Curve)	Varies	Good balance of precision and recall, but improvements needed for rare diseases.
Confusion Matrix Accuracy	High for major classes	Lower in traditional CNN models	High classification accuracy for Tomato Bacterial Spot and Healthy Leaves.
Misclassification Rate	Moderate	Higher in older models	YOLOv8 improves misclassification but still struggles with similar disease types.

Training Speed	Fast (Optimized for GPU)	Slower (Traditional CNNs)	Faster convergence with YOLOv8, reducing training time.
Inference Speed	Real-time	Slower	Suitable for real-time applications in plant disease monitoring.
Dataset Requirement	Moderate	High (for traditional CNNs)	YOLOv8 performs well even with moderate dataset sizes

Table 5. performance analysis

6.10 COMPARATIVE ANALYSIS OF YOLO VERSIONS

The comparative analysis of YOLO versions as shown in Table 6 evaluates the performance of YOLO v5, v7, and v8 on the plant disease detection task.

Model	mAP@50 (%)	Inference Time (ms)	Parameters
YOLO v5	92.4	12.5	7.0M
YOLO v7	94.1	10.2	6.2M
YOLO v8	96.3	8.9	5.5M

Table 6. comparative analysis of YOLO versions

6.11 MODEL EVALUATION RESULT

The model's evaluation results as shown in Table 7 summarize the overall performance of the YOLO v8 model on the plant disease detection task.

Metric	Value (%)
Precision	95.6
Recall	94.2
mAP@50	98.3

Table 7. model evaluation results

CHAPTER 7

CONCLUSION

The implementation of YOLO v8 for tomato leaf disease detection presents a significant advancement in precision agriculture. The proposed system efficiently detects and classifies plant diseases using deep learning-based object detection techniques, ensuring rapid and accurate diagnosis.

By leveraging YOLO v8, the system benefits from real-time processing, high accuracy, and robust performance in various lighting and environmental conditions. The dataset selection, preprocessing, model training, and evaluation steps ensure that the system can generalize well to new, unseen images, making it a reliable tool for farmers and agricultural experts.

The results obtained through the evaluation metrics such as precision, recall, F1-score, and mAP (mean Average Precision) indicate that YOLO v8 performs better than traditional machine learning models and earlier YOLO versions. The integration of preprocessing techniques, data augmentation, and hyperparameter tuning further enhances the model's effectiveness in disease detection.

Future improvements can focus on expanding the dataset, incorporating multi-modal data (such as thermal or hyperspectral imaging), and developing an integrated mobile application for real-world usability. Additionally, integrating Internet of Things (IoT) devices and cloud-based platforms can enhance scalability and accessibility for large-scale agricultural monitoring.

Overall, the system provides a cost-effective, scalable, and accurate solution for plant disease detection, reducing dependency on manual inspection and enabling proactive plant disease management. This technology holds great promise for improving crop yield, reducing losses, and ensuring food security in the long run.

CHAPTER 8

APPENDICES

CODE SNIPETS:

```
 def plot_sample_images(folder, num_samples=3):  
    """Plot sample images with bounding boxes from the dataset"""  
    img_dir = os.path.join(folder, "images")  
    label_dir = os.path.join(folder, "labels")  
  
    # Get a list of image files  
    img_files = [f for f in os.listdir(img_dir) if f.endswith(('.jpg', '.jpeg', '.png'))]  
  
    if not img_files:  
        print("No images found.")  
        return  
  
    # Randomly select files  
    selected_files = random.sample(img_files, min(num_samples, len(img_files)))  
  
    # Get class names from YAML  
    class_names = yaml_content.get('names', [])  
  
    fig, axes = plt.subplots(1, len(selected_files), figsize=(15, 5))  
    if len(selected_files) == 1:  
        axes = [axes]  
  
    for i, img_file in enumerate(selected_files):  
        # Load image  
        img_path = os.path.join(img_dir, img_file)  
        img = cv2.imread(img_path)  
        img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)  
        h, w, _ = img.shape  
  
        # Load corresponding label if exists  
        label_file = os.path.join(label_dir, os.path.splitext(img_file)[0] + '.txt')  
        title = os.path.basename(img_file)
```



```

# Draw bounding boxes
for line in lines:
    data = line.strip().split()
    class_id = int(data[0])
    x_center, y_center, width, height = map(float, data[1:5])

    # Convert YOLO format to pixel coordinates
    x1 = int((x_center - width / 2) * w)
    y1 = int((y_center - height / 2) * h)
    x2 = int((x_center + width / 2) * w)
    y2 = int((y_center + height / 2) * h)

    # Draw rectangle
    cv2.rectangle(img, (x1, y1), (x2, y2), (255, 0, 0), 2)

    # Add class name
    if class_id < len(class_names):
        label = class_names[class_id]
        title = f"class: {label}"
        cv2.putText(img, label, (x1, y1 - 10), cv2.FONT_HERSHEY_SIMPLEX, 0.5, (255, 0, 0), 2)

# Display image
axes[i].imshow(img)
axes[i].set_title(title)
axes[i].axis('off')

plt.tight_layout()
plt.show()

# Display sample images from training set
print("\nSample images from training set:")
plot_sample_images(TRAIN_PATH, num_samples=3)

```

Figure 8.1 Display Sample Images from training set

```

▶ def train_yolo_model(yaml_path, epochs=100, imgsz=640, batch_size=16):
    """Optimized YOLOv8 Training for Better Accuracy"""
    print("\n--- Starting Optimized YOLOv8 Training ---")

    # Clear GPU memory before training
    import torch, gc
    gc.collect()
    torch.cuda.empty_cache()

    # Load a larger YOLOv8 model for better accuracy
    model = YOLO('yolov8m.pt')

    # Train the model
    results = model.train(
        data=yaml_path,
        epochs=epochs,
        imgsz=imgsz,
        batch=batch_size,
        patience=10,
        workers=8,
        half=torch.cuda.is_available(),
        name='tomato_disease_model_optimized',
        verbose=True
    )

    print("--- Training Completed ---")
    return model

# Train the model (make sure YAML_PATH is correct)
model = train_yolo_model(YAML_PATH, epochs=100, batch_size=16)

```

Figure 8.2 Train the model

```

▶ def evaluate_model(model, data_yaml):
    """Evaluate the trained YOLOv8 model on the validation dataset."""

    print("\n--- Evaluating Model on Validation Set ---")

    # Run validation
    results = model.val(data=data_yaml)

    # Debug: Print all available attributes
    print("\nAvailable attributes in results:", dir(results))

    # Extract evaluation metrics
    try:
        if hasattr(results, 'box'):
            print(f"mAP@0.5: {results.box.map50:.4f}")
            print(f"mAP@0.5:0.95: {results.box.map:.4f}")
        else:
            print("Error: 'box' attribute not found in results.")
    except AttributeError:
        print("Error: Could not extract evaluation metrics. Check results object structure.")

    return results

# Run evaluation
val_results = evaluate_model(model, data_yaml="/content/My-First-Project-1/data.yaml")

```

Figure 8.3 Evaluate the model

```

import os
import cv2
import matplotlib.pyplot as plt

def test_on_specific_images(model, test_dir, image_list):
    """Test the model on specific images provided by the user"""

    test_img_dir = os.path.join(test_dir, "images")

    for img_file in image_list:
        img_path = os.path.join(test_img_dir, img_file)

        # Check if the file exists
        if not os.path.exists(img_path):
            print(f"⚠ File {img_file} not found in {test_img_dir}. Skipping.")
            continue

        print(f"\n--- Testing Model on {img_file} ---")

        # Run prediction
        results = model.predict(img_path, conf=0.25)

        # Get the image with predictions
        result_img = results[0].plot()

        # Display image
        plt.figure(figsize=(10, 8))
        plt.imshow(cv2.cvtColor(result_img, cv2.COLOR_BGR2RGB))
        plt.title(f"Predictions for {img_file}")
        plt.axis('off')
        plt.show()

        # Show prediction details
        print(f"\nPredictions for {img_file}:")
        boxes = results[0].boxes
        for i, box in enumerate(boxes):
            cls = int(box.cls[0].item())
            conf = box.conf[0].item()
            class_name = model.names[cls]
            print(f" - Object {i+1}: {class_name}, Confidence: {conf:.4f}")

# Example Usage: Provide specific image file names
image_list = ["085cbe78-1d5c-45eb-877f-f409526032d5__GH_HL-Leaf-469_JPG.rf.84c0dd63e48dcd54760f20a6533a214a.jpg"]
test_on_specific_images(model, TEST_PATH, image_list)

```

Figure 8.4 Test the Model

REFERENCES

1. Arathi, B., & Dulhare, U. N. (2023). Classification of cotton leaf diseases using transfer learning-denseNet-121. *Proceedings of Third International Conference on Advances in Computer Engineering and Communication Systems: ICACECS 2022*, 393–405.
2. Attallah, O. (2023). Tomato leaf disease classification via compact convolutional neural networks with transfer learning and feature selection. *Horticulture*, 9(149). doi: 10.3390/horticulturae9020149.
3. Bin Naeem, A., Senapati, B., Chauhan, A. S., Kumar, S., Gavilan, J. C. O., & AbdelRehim, W. M. F. (2023). Deep learning models for cotton leaf disease detection with VGG-16. *Int. J. Intelligent Syst. Appl. Eng.*, 11, 550–556.
4. Chen, Y., & Wu, Q. (2023). Grape leaf disease identification with sparse data via generative adversarial networks and convolutional neural networks. *Precis Agric.*, 24, 235–253. doi: 10.1007/s11119-022-09941-z.
5. Chen, Z., Ruhui, W., Yiyan, L., Chuyu, L., Siyu, C., Zhineng, Y., et al. (2022). Plant disease recognition model based on improved YOLOv5. *Agronomy*, 12(365). doi: 10.3390/agronomy12020365.
6. Chuanlei, Z., Shanwen, Z., Jucheng, Y., Yancui, S., & Jia, C. (2017). Apple leaf disease identification using genetic algorithm and correlation-based feature selection method. *Int. J. Agric. Biol. Eng.*, 10, 74–83.
7. Dhinesh, E., Jagan, A., et al. (2019). Detection of leaf disease using principal component analysis and linear support vector machine. *2019 11th International Conference on Advanced Computing (ICoAC)*, 350–355.
8. Eunice, J., Popescu, D. E., Chowdary, M. K., & Hemanth, J. (2022). Deep learning-based leaf disease detection in crops using images for agricultural applications. *Agronomy*, 12(2395).
9. Ghosal, S., Blystone, D., & Singh, A. (2021). A hybrid deep learning model for plant disease detection using CNN and transfer learning. *Computers and Electronics in Agriculture*, 190, 106491.
10. Mohanty, S. P., Hughes, D. P., & Salathé, M. (2016). Using deep learning for image-based plant disease detection. *Frontiers in Plant Science*, 7, 1419.